オペレーティングシステム
とLinuxの基本操作

Ubuntu Server 24.04 LTS

JDU Cyber Security / 2026年度教材

第1章 コンピューターとオペレーティングシステム

この章で答えられるようになる問い

- コンピューターは、どのような部品と入出力装置から構成されるか。
- パソコンとサーバーは、何が同じで何が異なるか。
- OSは、利用者、アプリケーション、ハードウェアの間で何をしているか。
- ユーザー空間とカーネル空間は、どのように分かれているか。
- Linuxシステムは、どのような順序で起動し、サービスを動かすか。
- ユーザー、ホスト、プロセスは、どのように識別されるか。

1.1 コンピューターを構成するもの

コンピューターは、入力された情報をプログラムに従って処理し、結果を出力または保存する機械である。ノートパソコン、デスクトップパソコン、スマートフォン、クラウド上の仮想サーバーは外見や用途が異なるが、基本的な構成は共通している。

- CPU: プログラムの命令を読み取り、計算や判断を実行する。
- メモリ: 実行中のプログラムと、そのプログラムがすぐに使うデータを一時的に置く。
- ストレージ（保存領域）: 電源を切った後も残すプログラム、設定、文書、データ、ログを保存する。SSDや仮想ディスクなどが該当する。
- 入力装置: 利用者の操作や外部の情報を受け取る。キーボード、マウス、タッチパネルなどがある。
- 出力装置: 処理結果を利用者へ伝える。ディスプレイ、スピーカー、プリンターなどがある。
- ネットワーク装置: ほかのコンピューターとデータを送受信する。

利用者がキーボードを押しただけでは、CPUやストレージが自動的に目的を理解するわけではない。入力をアプリケーションへ渡し、アプリケーションが必要とするCPU時間やメモリを割り当て、保存されたファイルを読み書きし、結果を画面やネットワークへ返す共通の仕組みが必要である。その中心を担うのがオペレーティングシステム（OS）である。

1.2 パソコンとサーバーは同じ種類のコンピューターである

パソコン（PC）は、通常は目の前の利用者がキーボード、マウス、ディスプレイを使って操作する。一方、サーバーは、ネットワークを通じてほかのコンピューターへ機能やデータを提供する役割を持つ。Webページを返すWebサーバー、データを保存・検索するデータベースサーバー、名前解決を行うDNSサーバーなどがある。

サーバーはパソコンとは別原理の機械ではない。どちらにもCPU、メモリ、ストレージ、ネットワーク、OS、アプリケーションがある。違いは主に用途、性能、接続方法、連続稼働の要件である。サーバーには常設のキーボードやディスプレイがないことも多く、管理者は別のコンピューターからネットワーク経由で操作する。AWS EC2のインスタンスも、AWSの設備内に作られた仮想的なコンピューターであり、本授業ではサーバーとして使用する。

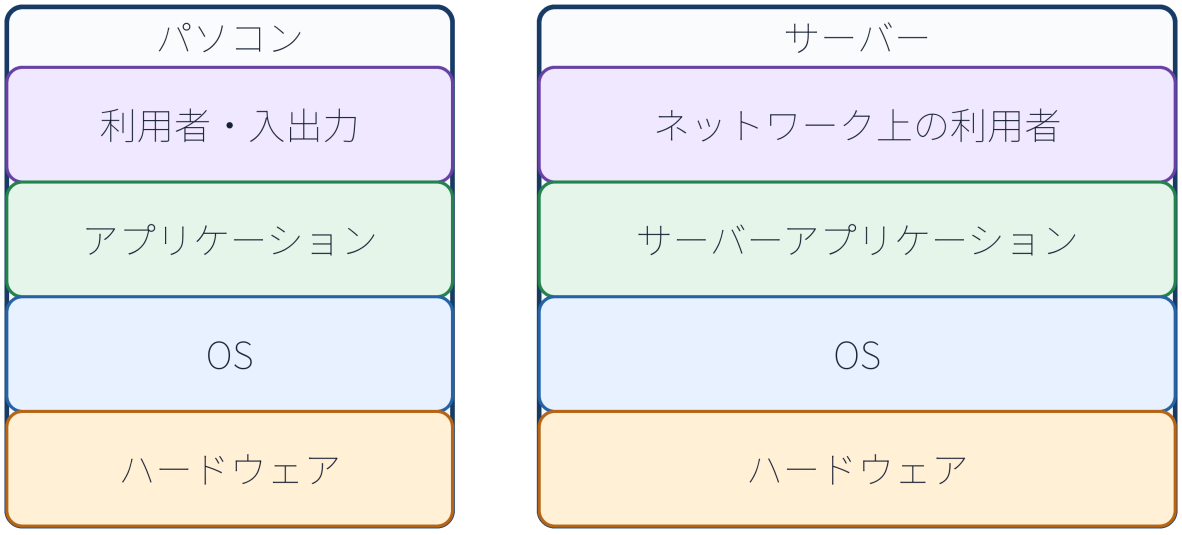


図1-1 パソコンとサーバーは、用途や操作方法が異なっても、アプリケーション、OS、ハードウェアという基本構成を共有する。

1.3 OSは何を管理するのか

複数のアプリケーションが同時に動くとき、それぞれがCPUやメモリを自由に独占したり、ほかの利用者のファイルを勝手に読んだりしては困る。OSは、ハードウェアとアプリケーションの間に入り、資源の利用を調整し、共通の利用方法と保護機能を提供する。

本書では、OSの主な働きを次の四つに整理する。

1. 実行の管理: プログラムをプロセスとして動かし、CPUを使う順序や実行状態を管理する。
2. メモリとファイルの管理: メモリをプロセスへ割り当て、ストレージをファイルとディレクトリとして利用できる形にする。
3. 利用者と保護の管理: ユーザー、グループ、アクセス権（パーミッション）を使い、誰が何を読めるか、変更できるか、実行できるかを判断する。
4. 通信の管理: プロセスがネットワークを利用するためのソケットなどの共通機能を提供する。

OSが共通機能を提供するため、アプリケーションの開発者は、機種ごとにCPUやディスクを直接制御する処理をすべて書き直す必要がない。

1.4 GUI、CLI、コマンド

人がコンピューターへ操作を伝える方法をユーザーインターフェースという。代表的なものがGUIとCLIである。

- GUI（Graphical User Interface）: ウィンドウ、アイコン、メニュー、ボタンなどを画面に表示し、主にマウスやタッチ操作で指示する。
- CLI（Command Line Interface）: 文字で命令を入力し、文字で結果を受け取る。

CLIで入力する個々の命令をコマンドという。コマンド名に、必要に応じてオプションや処理対象を続ける。たとえば、`cat`はファイルの内容を順に読み取り、標準出力へ表示するコマンドである。

```
cat /etc/os-release
```

この例では、catがコマンド名、/etc/os-releaseが読み取るファイルのパスである。入力された文字列はシェルによって解釈され、catプログラムが起動される。catはOSへファイルの読み取りを依頼し、OSはアクセス権を確認してから内容を渡す。コマンド、ターミナル、シェルの違いと基本操作は第3章で詳しく扱う。

1.5 ユーザー空間とカーネル空間

Linuxを使用しているシステムは、概念上、ユーザー空間（user space）とカーネル空間（kernel space）に分けて考えられる。

カーネルはOSの中核であり、CPU、メモリ、デバイス、ファイルシステム、プロセス、アクセス権、ネットワークを管理する。カーネルは強い権限を持つため、通常のアプリケーションから分離されたカーネル空間で動く。

GUIを構成するプログラム、ターミナル、シェル、コマンド、Webサーバー、データベースなどは、通常はユーザー空間で動く。GUIとCLIは「別の空間」ではなく、どちらもユーザー空間に存在する操作方法と関連プログラムである。アプリケーションもユーザー空間で動き、必要な処理をカーネルへ依頼する。

この境界を通ってカーネルの機能を呼び出す仕組みをシステムコール（system call）という。本書ではシステムコールの番号やプログラミング方法までは扱わない。「ユーザー空間のプログラムが、保護されたカーネル機能を利用するための窓口」と理解すればよい。

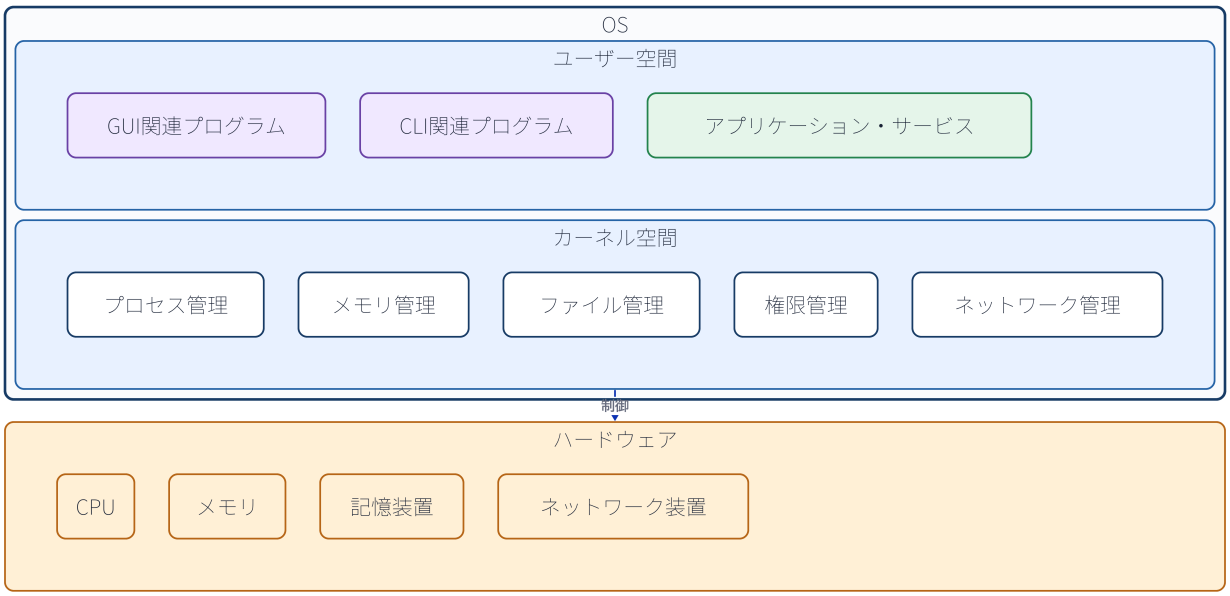


図1-2 広い意味でのOS環境は、ユーザー空間で動く各種プログラムと、ハードウェアを管理するカーネル空間から構成される。

厳密には、「OS」という語をカーネルだけに近い意味で使う場合と、基本コマンドや管理プログラムを含むシステム全体の意味で使う場合がある。本書では、両者を区別する必要がある箇所では「カーネル」と「ユーザー空間のプログラム」という名称を使う。

1.6 LinuxとUbuntu

Linuxは本来カーネルの名称である。一方、実際に利用できるシステムを作るには、カーネルに加えて、基本コマンド、シェル、ライブラリ、パッケージ管理、サービス管理などが必要になる。これらを組み合わせて導入・更新できる形にまとめたものをLinuxディストリビューションという。

Ubuntu、Debian、FedoraなどはLinuxディストリビューションの例である。本授業では実習環境を統一するためにUbuntu Server 24.04 LTSを使用するが、学習の中心はUbuntuという製品名ではなく、Linuxシステムに共通す

る基本概念と操作である。ディストリビューションの歴史と違いは第2章で扱う。

1.7 電源投入からサービスが動くまで

ストレージに保存されているプログラムは、置かれているだけでは処理を行わない。プログラムがメモリへ読み込まれ、CPUによって実行されている状態の存在をプロセスという。OSは各プロセスを区別するために、PID（プロセスID）という番号を割り当てる。

Linuxシステムの起動は、概略として次の順序で進む。

1. 電源投入後、ファームウェアがCPU、メモリ、ストレージなどを初期化する。
2. ブートローダーがLinuxカーネルをメモリへ読み込む。
3. カーネルがメモリ、デバイス、ファイルシステムなどを利用できる状態にする。
4. カーネルが、ユーザー空間で最初に動く管理プロセスを起動する。
5. Ubuntu Server 24.04では、その管理プロセスとして`systemd`が動き、PID 1を持つ。
6. `systemd`が設定と依存関係を読み、ネットワーク、SSH、ログ管理、Webサーバー、データベースなどに必要なサービスを起動・監視する。

サービスとは、利用者が画面から毎回起動しなくても、バックグラウンドで継続的な機能を提供するよう管理されるプログラムである。サービスを起動すると、実際の処理を行う一つ以上のプロセスが動く。たとえばデータベースサーバーでは、データベースのサービスが管理対象となり、その実体であるプロセスがメモリ上で動き、必要に応じてファイルを読み書きし、ネットワークからの要求を待ち受ける。

`systemd`はデータベースやWeb処理そのものを実行するプログラムではない。どのサービスを、いつ、どの順番で、どのユーザーとして起動するかを管理する。PID、プロセスの親子関係、シグナルは第8章、`systemd`、サービス、ユニットは第10章で詳しく扱う。

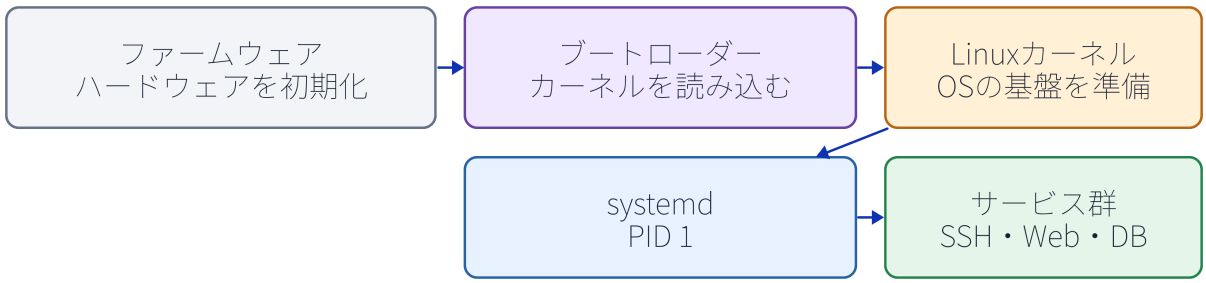


図1-3 カーネルが最初のユーザー空間プロセスを起動し、systemdが必要なサービスプロセスを順序付けて起動する。

1.8 ホストとユーザーの識別

ネットワークへ接続され、独立したOS環境として動作するコンピューターまたは仮想マシンを、ホスト（host）と呼ぶ。ホストには、それを識別するためのホスト名（hostname）が設定される。ホスト名はコンピューター側の名前であり、利用者の名前ではない。

一つのホストには複数のユーザーアカウントを登録できる。OSは、操作を行っているユーザーと所属グループを数値のIDで管理する。ユーザーを識別する番号をUID（user ID）、グループを識別する番号をGID（group ID）という。`id`コマンドは、現在のユーザー名、UID、主グループ、補助グループを確認するために使う。

本演習では、学生PC、AWS CloudShell、Ubuntu EC2という複数のホストを使用する。画面上では同じような文字入力欄に見えても、それぞれが別のOS、ユーザー、プロセス、ファイルを持つ。現在のユーザー名、ホスト名、作業ディレクトリを確認することで、どのホストのどの場所を操作しているかを判断できる。

```
id
hostname
pwd
```

idはユーザーとグループの識別情報、hostnameはホスト名、pwdは現在の作業ディレクトリを表示する。ユーザー管理は第6章、ファイルとパスは第4章、CloudShellとUbuntuの接続関係は第12章で詳しく扱う。

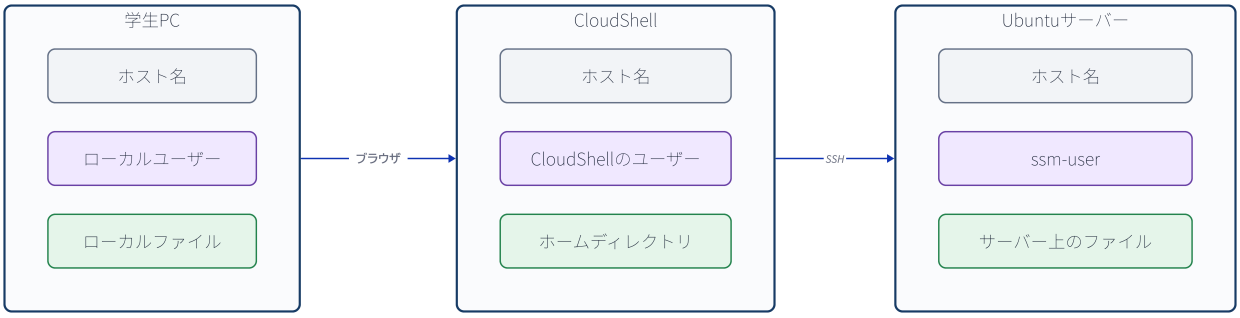


図1-4 各ホストは独立したOS環境を持ち、その中でユーザー、プロセス、ファイルが個別に管理される。

章末問題

問1 パソコンとサーバーは、何が同じで何が異なるか。

問2 GUI、CLI、ユーザー空間、カーネル空間はどのような関係にあるか。

問3 プログラム、プロセス、サービスの違いは何か。

問4 PID 1のsystemdは何をするか。

問5 ホスト名とユーザー名は何が違うか。

章末解答

問1

答え: どちらもCPU、メモリ、ストレージ、ネットワーク、OS、アプリケーションを持つコンピューターである。パソコンは目の前の利用者による対話操作を主な用途とし、サーバーはネットワークを通じてほかのコンピューターへ機能やデータを提供することを主な役割とする。

問2

答え: GUIとCLIは利用者が操作を伝える方法であり、それを実現するプログラムは通常ユーザー空間で動く。ユーザー空間のプログラムは、システムコールを通じてカーネル空間の機能を利用する。カーネルはハードウェアとOS資源を管理する。

問3

答え: プログラムはストレージに保存された命令群である。プロセスは、そのプログラムがメモリへ読み込まれて実行されている存在である。サービスは、継続的な機能を提供するようsystemdなどによって管理される単位であり、その実体として一つ以上のプロセスが動く。

問4

答え: Ubuntu Server 24.04では、カーネルが最初に起動するユーザー空間の管理プロセスがsystemdであり、PID 1を持つ。systemdは設定と依存関係に従って各種サービスを起動、停止、監視する。

問5

答え: ホスト名はOSが動いているコンピューターまたは仮想マシンを識別する名前である。ユーザー名は、そのホスト上で操作権限を持つアカウントを識別する名前である。一つのホストに複数のユーザーを登録できる。

参考資料

- Linux Kernel Organization, The Linux Kernel documentation（2026-09-21確認）
- Linux man-pages, intro(2) - introduction to system calls（2026-09-21確認）
- Ubuntu Manpages, boot(7) - system bootup process（2026-09-21確認）
- Ubuntu Server documentation, Ubuntu Server documentation（2026-09-21確認）

第2章 Unixから現在のOSへ

なぜ歴史を学ぶのか

歴史を学ぶ目的は年号の暗記ではない。Linux、macOS、Windowsの間で似た用語や異なるコマンドが使われる理由を理解し、「LinuxはUnixのソースコードをそのまま受け継いだ」「macOSはLinuxの一種である」「WindowsもUnixから派生した」といった誤解を避けるために学ぶ。

2.1 Unixが示した考え方

Unixは1969年にベル研究所（Bell Labs）で開発が始まった。その後の発展を通じて、階層型ファイルシステム、プロセス、マルチユーザー（複数ユーザー）、小さなツール（small tools）の組み合わせなど、現在のOSにもつながる基本設計思想を広めた。



写真2-1 初期Unixの開発に使われた機種DEC PDP-7の保存機。2018年、米国の博物館で撮影。撮影: ComputerGeek7066、Wikimedia Commons、CC BY-SA 4.0（無加工）。

写真は当時のベル研究所の実機を撮影したものではない。PDP-7は1960年代に製造されたコンピューターで、初期Unixが動いた機種である。

本書の演習との接点は明確である。ファイルをディレクトリの階層構造で扱う。プログラムをプロセスとして観測する。ユーザーとグループで利用権限を分ける。`grep`の出力を別のコマンドやファイルへつなぐ。これらはUnix系システムで長く培われてきた操作モデルである。

「Unix」は単一のコマンド集の名称ではない。また、歴史上のUnixから影響を受けたシステムが、すべて同一のソースコードや正式なUNIX商標認証を持つわけでもない。一般にUnixに似た設計やインターフェースを持つシステムをUnix系（Unix-like）と呼ぶ。正式な商標・製品認証の話題と、設計思想上の分類は明確に分けて考える必要がある。

2.2 GNUと自由に利用できるUnix互換環境

GNUプロジェクト（GNU Project）は1983年に発表され、Unixと互換性のある自由なソフトウェアシステムの構築を目指した。シェル、コンパイラ、基本ユーティリティなど多くのプログラムが開発された。現在の多くのLinuxディストリビューションでは、Linuxカーネルの周囲でGNUのツール群を含む多様なソフトウェアが動作している。

シェル（shell）とは、利用者が入力した文字列を読み取り、コマンドとして解釈し、必要なプログラムを起動するユーザー空間のプログラムである。シェルは、単に一つのコマンドを起動するだけではない。引数の区切りを解釈する。入出力のリダイレクトやパイプを組み立てる。環境変数を展開する。複数のコマンドを書いたシェルスクリプトを順番に実行する。このように、利用者と多数のコマンドの間で操作を取りまとめる役割を持つ。

BashはGNUプロジェクトが開発するシェルの一つであり、名称は「Bourne-Again Shell」に由来する。Bash自体はLinuxカーネルでも、ターミナル画面でもない。Ubuntuのターミナルで入力した文字列をBashが解釈し、`ls`や`cp`などのプログラムを起動する。`cd`のようにシェル自身の状態を変える処理は、Bashに組み込まれたコマンドとして実行される。ターミナル、シェル、コマンドの違いは第3章で詳しく扱う。

ここで大切なのは、カーネル単体では学生が操作するシステム全体にはならないという点である。たとえばBashはシェルであり、GNU coreutilsは`ls`や`cp`などの基本コマンド群を提供する。さらにパッケージマネージャー、systemd、ライブラリ、設定ファイル、各種アプリケーションが組み合わさることで初めて実用的なシステムになる。

2.3 Linuxは独立に実装されたカーネル

Linuxカーネルは1991年に開発が始まった。Unixに似た機能とインターフェースを持つことを目指して開発されたが、歴史上のUnixのソースコードから直接枝分かれしたものではない。したがって、OSの系譜図ではUnixからLinuxへ「ソースコードを継承した実線」を引くのではなく、設計上の影響や互換性の目標を示す破線などで表現する。

Linuxカーネルは、プロセス、メモリ、ファイルシステム、デバイス、ネットワークなどの低レイヤーを担当する。UbuntuはLinuxカーネルに、多数のユーザー空間のツール群、パッケージ、初期設定、リリース管理方針を組み合わせたディストリビューションである。

一方、Windows NT系はUnixから枝分かれしたOSではなく、Unix系とは別の系統として設計された。Windowsの`cmd.exe`やPowerShellもシェルであるが、Bashとは構文、組み込みコマンド、パス表記、管理方法が異なる。名前が似たコマンドや、複数のOSで動く外部プログラムも存在するが、Unix系とWindowsで同じコマンドがそのまま利用できるとは考えない方がよい。BashはLinuxそのものではなく、LinuxやmacOSなどでも利用できるシェルの一つである。

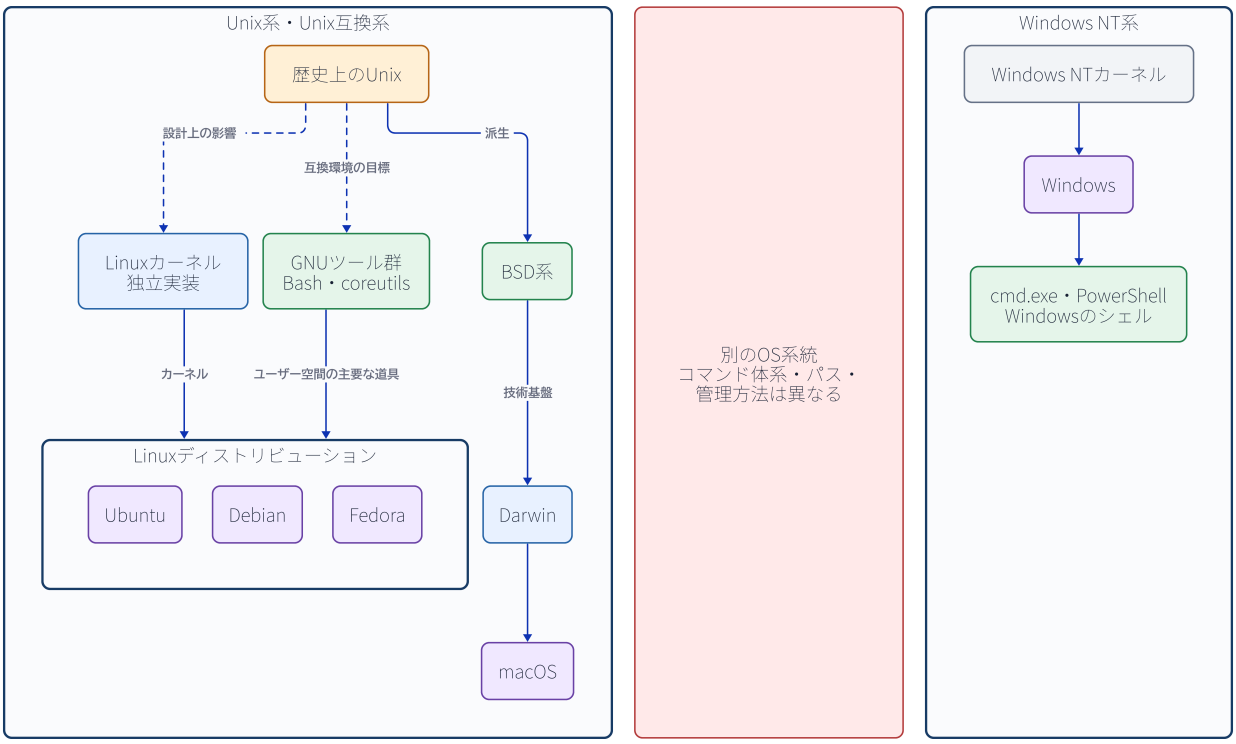


図2-1 Unix系・Unix互換系とWindows NT系は別の系統であり、コマンド体系の互換性を前提にできない。 LinuxはUnixに似たインターフェースを持つ独立実装であり、macOSはLinuxの派生ではない。

2.4 ディストリビューションが選択と統合を行う

Linuxカーネルを利用するシステムにはUbuntu、Debian、Fedoraなど複数のディストリビューションが存在する。各ディストリビューションは、採用するソフトウェア、パッケージ形式、更新方針、初期設定、サポート期間などを独自に決定する。

本授業では環境をUbuntu Server 24.04 LTSに統一する。ほかのディストリビューションでも同じ`ls`や`grep`を利用できる場合が多いが、パッケージ管理コマンド、設定ファイルの配置場所、セキュリティ機能、サービスの初期設定などが異なる場合がある。Red Hat/RHEL向けの手順をUbuntuへ無条件に当てはめてはならない。

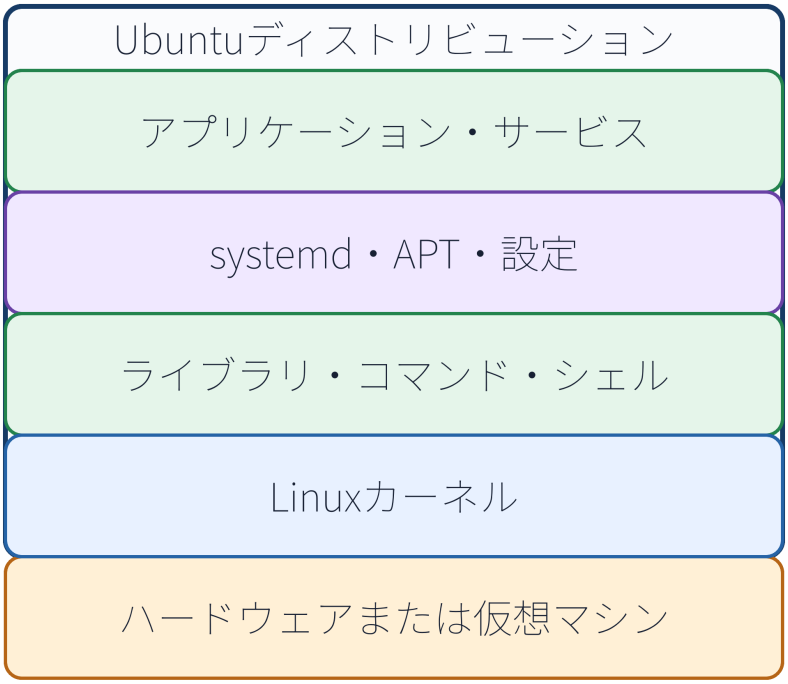


図2-2 Linuxカーネル、ライブラリ・基本コマンド、システム管理、パッケージ、アプリケーションをディストリビューションが統合する構造。 UbuntuとLinuxを完全な同義語として混同しないための構成図である。

2.5 macOSはDarwinを基盤に持つ

現在のmacOSはDarwinを中核（カーネル基盤）に持つ。Appleの公式資料では、DarwinはMach、BSD、Apple独自の技術などから構成されていると説明されている。BSD由来のプロセスモデル、アクセス権体系、ネットワーク機能などを備えているため、ターミナル上でUnix系に類似したコマンドやパスを利用できる。

ただし、macOSはLinuxディストリビューションではない。Linuxカーネルを中核にしておらず、独自のGUIシステムやApple固有のフレームワークなど、Darwin単体では構成されない領域も併せ持つ。また、古いClassic Mac OSと、Mac OS X以降のDarwin基盤の系譜も区別して理解する必要がある。

2.6 Windowsは別の系統として理解する

初期の一般消費者向けWindowsにはMS-DOSを土台にした系列（Windows 95/98など）が存在した。一方、現在のWindowsへ直接つながるWindows NT系列は、それらとは別にゼロから設計されたシステムである。Windows 2000、XP、そして現代のWindowsに至る流れを、Unixから単純に派生した系譜として描いてはならない。

Windowsにもカーネルモードとユーザーモード、プロセス、ファイルアクセス権、サービス、ネットワークソケット、CLIが存在する。これは現代のOSが解決すべき技術的課題に共通性があるためであり、Unixの子孫であることの証明ではない。PowerShellを利用できることや、Windows Subsystem for Linux（WSL）が動作することも、Windowsカーネル自体がLinuxになったことを意味するわけではない。

2.7 共通点と相違点をどう見るか

観点	Ubuntu/Linux系	macOS	Windows
中核	Linuxカーネル	Darwin（Mach・BSD等）	Windows NT系カーネル
主な標準CLI	シェルとUnix系ツール群	シェルとBSD/Unix系ツール群	PowerShell、cmd等
パスの例	/home/user	/Users/user	C:\Users\user
ソフトウェア導入	ディストリビューションのパッケージ管理	App Store、インストーラー、パッケージ管理ツール等	Microsoft Store、インストーラー、パッケージ管理ツール等
本授業での扱い	実習対象	比較対象	比較対象

同じ目的を持つ機能であっても、具体的なコマンド名、初期設定、アクセス権モデルの詳細な挙動はOSごとに異なる。本授業ではUbuntuを用いてOSの基本原理を学び、将来ほかのOSへ移行する際にも、「同じコマンドを探す」のではなく、「同じ目的を担う機能や仕組みを探す」姿勢を身に付けることが重要である。

章末問題

問1 LinuxとUbuntuの違いを、「カーネル」と「ディストリビューション」という用語を用いて説明せよ。

問2 シェルとは何か。Bashを例に説明せよ。

問3 macOSをLinuxディストリビューションと呼べない理由を説明せよ。

問4 Windowsにもプロセスやアクセス権が存在することは、WindowsがUnixから直接派生した証拠になるか。理由とともに答えよ。

章末解答

問1

答え: Linuxはプロセス、メモリ、デバイスなどを管理するカーネルである。UbuntuはLinuxカーネルに、GNUツール群、シェル、ライブラリ、パッケージ管理、設定などを組み合わせたLinuxディストリビューションである。

問2

答え: シェルは、利用者が入力した文字列をコマンドとして解釈し、プログラムの起動、引数の受け渡し、リダイレクト、パイプ、変数展開、スクリプト実行などを行うユーザー空間のプログラムである。BashはGNUプロジェクトが開発するシェルの一つであり、Linuxカーネルそのものではない。

問3

答え: macOSはDarwinを基盤とし、その中核にはMachやBSD由来の技術が使われている。Linuxカーネルを中核としていないため、Linuxディストリビューションではない。

問4

答え: 証拠にはならない。プロセスやアクセス権は、現代のOSが共通して必要とする機能である。Windows NT系はUnix系とは別の系統であり、`cmd.exe`やPowerShellの構文、コマンド、パス、管理方法もBashを中心とするUnix系環境とは異なる。

参考資料

- Dennis M. Ritchie, The Evolution of the Unix Time-sharing System（2026-09-18確認）
- Bell Labs, A history of computing at Bell Research Laboratories (1937-1975)（2026-09-24確認。初期UnixとPDP-7の関係）
- Wikimedia Commons, DEC PDP-7.jpg（2026-09-24確認。ComputerGeek7066撮影、CC BY-SA 4.0。2018年に保存機を撮影）
- GNU Project, Initial Announcement（2026-09-18確認）
- GNU Project, What is a shell?（2026-09-21確認）
- GNU Project, What is Bash?（2026-09-21確認）
- Linux Kernel Documentation, Introduction（2026-09-18確認）
- Apple, Kernel Architecture Overview（2026-09-18確認。旧archiveであるため歴史・構成の確認に限定）
- Microsoft, Windows NT Build History（2026-09-18確認。歴史資料）
- Microsoft Learn, Windows commands（2026-09-21確認）

第3章 GUI・CLI・ターミナル・シェル

3.1 操作画面と処理主体を分ける

GUI（グラフィカルユーザーインターフェース）は、ウィンドウ、アイコン、メニュー、ポインタなどを使って視覚的に操作する方法である。CLI（コマンドラインインターフェース）は、文字列（テキスト）でコマンドを入力し、文字列の結果を受け取る操作方法である。どちらも人間がコンピューターへ指示を出すための入口であり、GUIが内部で必ずCLIへ変換されなければ動かない、という従属関係ではない。

ターミナル（terminal）は、文字を入力し結果を表示する画面・接続環境である。シェル（shell）は、その入力された文字列を読み取り、コマンドを解釈して実行するプログラムである。本演習環境で主に使用するシェルは Bash である。

```
人間（操作者） → ターミナル → Bash（シェル） → コマンドのプロセス → カーネルの機能
```

ターミナルとシェルは同じものではない。ターミナルは文字をやり取りする場所であり、シェルは入力された文字列の意味を判断して処理するプログラムである。

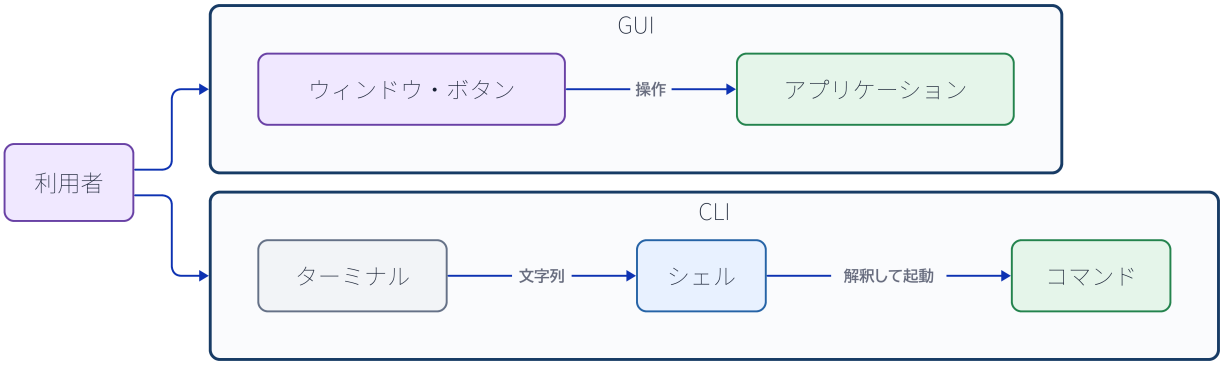


図3-1 GUIとCLIは並列の操作入口。CLIではターミナルとシェルを明確に区別する。

3.2 プロンプトは入力位置を示す

シェルがユーザーからの入力を待っているときに表示する文字列をプロンプト（prompt）という。プロンプトにはユーザー名、ホスト名、現在位置（カレントディレクトリ）などが表示されることが多いが、設定によって表示内容は異なる。

教材中の次のような`$`はプロンプトの表示例であり、ユーザー自身が入力する文字ではない。

```
$ pwd
/home/ssm-user
```

入力するコマンドは`pwd`であり、その次の行は出力例である。本書のコマンド例では、入力する部分と出力を区別するために`$`を表示する場合がある。

3.3 コマンド、引数、オプション

基本形は次のように読む。

```
コマンド  オプション  引数
ls     -l       /srv
```

lsは実行するコマンド、-lは詳細表示を指定するオプション、/srvは対象を指定する引数（argument）である。短いオプションを組み合わせてls -laのように指定できるコマンドも多いが、すべてのコマンドで同じアルファベットが同じ意味を持つとは限らない。たとえばgrep -nは行番号の表示を意味するが、tail -n 5は表示する行数を指定する。

コマンドラインでは空白が要素の区切りとして扱われる。そのため、空白を含む一つの文字列をひとまとまりの引数として渡すには、クォート（引用符）で囲む必要がある。

```
printf '%s\n' 'JDU SSH transfer test'
```

クォート、変数展開、リダイレクトについては第5章で詳しく解説する。

3.4 シェル組み込みコマンドと外部コマンド

シェルへ入力できるコマンドには、大きく分けてシェル組み込みコマンドと外部コマンドがある。

シェル組み込みコマンド（shell builtin）は、Bash自身に用意されている機能である。別の実行ファイルを探して起動する必要がない。たとえばcdは、現在動作しているBash自身の作業ディレクトリを変更するための組み込みコマンドである。

外部コマンドは、ストレージ上に保存されたプログラムをシェルが探して起動するものである。ls、mkdir、grepなどは、Ubuntuで最初から利用できることが多いが、Bashの組み込み機能ではなく外部プログラムである。つまり、最初から使えることとシェルに組み込まれていることは別の分類である。

外部コマンドの中には、OSの導入時から用意されているものと、必要になったときに追加導入するものがある。たとえばpython3はPythonプログラムを実行する外部コマンドであり、環境にPythonが導入されている場合だけ利用できる。Ubuntuの構成によっては最初から存在するが、存在しない場合はパッケージとして導入する。パッケージの導入方法は第9章で扱う。

```
type cd
type mkdir
type python3
```

typeは、入力した名前をBashがどのように解釈するかを表示する。組み込みコマンドならその事実が表示され、外部コマンドなら実行ファイルの場所が表示される。見つからない場合は、その名前のコマンドを現在の環境では利用できない。

typeの結果にエイリアス（alias）と表示されることもある。エイリアスとは、長いコマンドやよく使う指定に、短い別名を付けるシェルの機能である。たとえば環境によってはllという名前にls -alFなどが登録されている。エイリアスの有無や内容は環境ごとに異なる。

外部コマンドが入力されると、シェルはPATHという設定に登録されたディレクトリを順に調べ、同じ名前の実行ファイルを探す。実際の保存場所を最初から暗記する必要はない。まずtypeで、組み込みコマンド、外部コマンド、エイリアスのどれとして認識されているかを確認する。

3.5 シェルがコマンドを実行する流れ

シェルは入力された文字列をそのままカーネルへ渡しているわけではない。内部では大まかに次の処理を順番に行っている。

1. 文字列をコマンド名、オプション、引数に分割・解釈する。
2. クォート、環境変数、コマンド置換$(...)、ワイルドカードなどを規則に従って展開する。
3. 組み込みコマンドであればシェル内部で処理し、外部コマンドであれば実行ファイルを探索する。
4. リダイレクトやパイプなど、必要な入出力ストリームの接続を準備する。
5. プロセスを開始し、フォアグラウンドで終了を待つか、バックグラウンドで継続動作させる。
6. プロセスの終了ステータスを受け取る。

この流れを知っていると、問題が起きた場所を切り分けやすくなる。コマンドが見つからない場合は、名前の解釈や外部プログラムの探索を確認する。期待した対象が処理されない場合は、オプションや引数の解釈を確認する。入出力やプロセスについては、第5章と第8章で詳しく扱う。

3.6 困ったときによく使う操作と調査コマンド

CLIで現在位置やコマンドの意味が分からなくなった場合は、推測で操作を続けず、次の操作で状況を確認する。

操作・コマンド	確認できること
Ctrl+C	現在実行中の処理へ停止を求め、プロンプトへ戻る。
pwd	現在作業しているディレクトリを表示する。
cd ディレクトリ	作業するディレクトリを移動する。
ls	現在のディレクトリにあるファイルやディレクトリを表示する。
grep '文字列' ファイル	ファイルから指定した文字列を含む行を探す。
type コマンド名	その名前が組み込み、外部コマンド、エイリアスのどれとして認識されるか調べる。
help コマンド名	Bashの組み込みコマンドの説明を表示する。
exit	現在のシェルを終了し、そのシェルを開始する前の状態へ戻る。

pwd、cd、lsは「どこで何を操作しているか」を確認する基本である。grepは長い文章やログから必要な行を探すときに使う。typeとhelpは、コマンド自体の種類や使い方を調べるために使う。なお、grepやlsは調査に便利な外部コマンドであり、すべてがシェル組み込みコマンドという意味ではない。

3.7 マニュアルとヘルプを活用する

外部コマンドのマニュアルはmanコマンド、Bashの組み込みコマンドはhelpコマンドが確認の入口となる。

```
man ls
help cd
grep --help
```

マニュアルではNAMEでコマンドの概要・目的、SYNOPSISで構文形式、DESCRIPTIONで詳細な動作仕様、OPTIONSで各オプションの意味を確認する。書式内の角括弧[ ]は、多くの場合「省略可能」であることを示す表記規則であり、その括弧記号自体を入力するわけではない。なお、manの閲覧画面はqキーで終了できる。

3.8 終了ステータスで成否を判断する

コマンドは画面へのテキスト出力とは別に、処理結果を表す小さな整数値を終了ステータス（終了コード、exit status）としてシェルへ返す。Unix系システムの共通規約として、`0`は「成功・正常終了」、`0`以外は何らかの「失敗・不成功・エラー」を意味する。

```
test -e /etc/os-release
echo $?
```

`$?`は直前に実行されたコマンドの終了ステータスを保持する特殊な変数である。別のコマンドを実行すると即座に上書きされてしまうため、確認したいコマンドの直後に参照する必要がある。また、`0`以外の値がすべて同じ原因を指すとは限らない。たとえば`grep`は「パターンに一致する行がなかった場合（1）」と「ファイルが存在しないなどの実行エラー（2）」で異なる値を返す。画面に表示されたメッセージだけでなく、システムが何を成功・失敗の判定基準としているかを確認することが重要である。

章末問題

問1 ターミナルとシェルの違いを、それぞれの役割に着目して説明せよ。

問2 `cd`がシェルの組み込みコマンドでなければならない理由を説明せよ。

問3 「最初から使えるコマンド」と「シェル組み込みコマンド」が同じ意味ではない理由を説明せよ。

問4 コマンドの種類を調べる方法を答えよ。

章末解答

問1

答え: ターミナルは文字を入力し結果を表示する場所・接続環境である。シェルは入力された文字列を解釈し、組み込み機能を処理したり外部プログラムを起動したりするプログラムである。

問2

答え: `cd`は、現在動作しているシェル自身の作業ディレクトリを変更する必要があるためである。別の外部プロセスだけが移動しても、元のシェルの現在位置は変わらない。

問3

答え: 最初から使える外部プログラムも存在するためである。たとえば`mkdir`はUbuntuで最初から使えることが多いが、Bashの組み込みではなく外部コマンドである。一方、`cd`はBashの組み込みコマンドである。

問4

答え: `type コマンド名`を使用する。組み込みコマンド、外部コマンド、エイリアスのどれとして認識されているかを確認できる。

参考資料

- Ubuntu Server documentation, Welcome to the terminal（2026-09-18参照。現行ページ構成は制作時に再確認）
- GNU Project, Bash Reference Manual（2026-09-18は本文取得timeout。執筆内容はUbuntu 24.04実機の`help`と`man bash`でも確認する）

第4章 ファイル、ディレクトリ、パス、編集

4.1 ファイルには内容とメタデータがある

ファイルにはデータ本文だけでなく、名前、種別、所有者（owner）、グループ（group）、アクセス権（パーミッション）、サイズ、最終更新時刻などのメタデータ（metadata）が存在する。`cat` で表示される内容が完全に同一の2つのファイルであっても、所有者やアクセス権が異なればOS上では別の状態として扱われる。

```
ls -l /srv/jdu-share/README.txt
stat /srv/jdu-share/README.txt
```

`ls -l` はディレクトリ内の主要情報を一覧形式で表示する。`stat` は指定した単一対象のメタデータを詳細に表示する。

4.2 ディレクトリは名前と対象を結び付ける

ディレクトリを「ファイルを箱の中に物理的に詰め込む入れ物」という比喩だけで理解すると、実態とずれて不正確になる。ディレクトリの本質は、ファイル名や別のディレクトリ名を実体と対応付け、階層構造を構築する特殊なファイルである。`/srv/jdu-share/README.txt` というパスは、最上位のルートディレクトリ `/` から出発し、`srv`、`jdu-share`、`README.txt` という名前を順番にたどる位置指定である。

ディレクトリツリーの例（抜粋）

```
/
├── home
│   └── ssm-user
│       └── jdu-lab
│           └── m1
│               └── errors.txt
├── etc
└── srv
    └── jdu-share
        └── README.txt
```

図4-1 `tree`コマンドの表示に近い階層図。枝を順にたどるとファイルの位置が分かる。

Linuxでは、システム全体がルート `/` を頂点とする単一のディレクトリツリー（階層構造）として構成される。物理的に異なるディスクや仮想ファイルシステムも、特定のディレクトリへ結合してシステム全体の一部として見せることができる。この接続操作をマウント（mount）という。本章では概念の理解にとどめ、ディスクのフォーマットやマウントの具体的な操作は後続のストレージ単元で扱う。

ルートディレクトリは、ディレクトリツリー全体の出発点 `/` である。ホームディレクトリは、各ユーザーが自分のファイルを置く場所である。たとえば `ssm-user` のホームディレクトリは `/home/ssm-user` であり、`/` の下にある `home`、その下にある `ssm-user` と順にたどって到達する。`/home` は複数ユーザーのホームディレクトリを置く親ディ

レクトリで、ssm-user 本人のホームディレクトリではない。/ と /home/ssm-user は役割も位置も異なる。名前が似ている /root も、ルートディレクトリ / とは別の場所である。

4.3 絶対パスと相対パス

先頭がスラッシュ / で始まるパスは絶対パス（absolute path）である。現在どのディレクトリにいても、常に同じ場所を一意に指し示す。

```
/srv/jdu-share/README.txt
```

先頭がスラッシュ / で始まらないパスは相対パス（relative path）であり、現在のカレントディレクトリ（作業ディレクトリ）を基準として位置を指定する。

```
cd /srv/jdu-share
cat README.txt
```

pwd は現在のカレントディレクトリを表示する。. は「現在のディレクトリ」、.. は「1つ上の親ディレクトリ」、~（チルダ）は「現在のユーザーのホームディレクトリ」へシェルによって自動展開される。

```
cd ~/jdu-lab/m1
pwd
cd ..
pwd
```

~ は現在のユーザーのホームディレクトリを表し、ルートディレクトリ / を表す記号ではない。~ を含むパスが指す場所は、操作中のユーザーやホストによって変わる。たとえば ssm-user の ~ が /home/ssm-user であっても、su - jduops でユーザーを切り替えると、~ は jduops のホームディレクトリ（/home/jduops など）を指す。別のホストに接続した場合も、そのホスト上で操作しているユーザーのホームディレクトリを指す。現在の場所は pwd、現在のホームディレクトリは printf '%s\n' "$HOME" で確認できる。HOME はホームディレクトリの場所を保持する環境変数であり、変数の仕組みは第5章で説明する。

4.4 Ubuntuでよく見るディレクトリ

パス	本教材での役割
/home	通常ユーザーのホームディレクトリが配置される
/etc	システム全体や各種サービスの設定・識別情報ファイル群。/etc/os-release もここに配置される
/usr/bin	多くの一般的なコマンドの実行ファイルが格納される
/var/log	システムやサービスのログファイルが配置される。すべてのログがここにあるとは限らない
/srv	システムやサービスが提供するデータを配置する領域。本演習では共有フォルダやWebコンテンツに使用する
/tmp	一時ファイル用のディレクトリ。誰でも読み書きできるが、恒久的な保存先ではない
/proc	カーネルがプロセスやシステムの状態をファイルの形式で見せる仮想ファイルシステム

この配置表は「絶対にこのディレクトリにしか置いてはならない」という固定規則ではない。ディストリビューションの方針やアプリケーションの設計によって細部は異なる場合がある。

4.5 作成、コピー、移動、削除

```
mkdir -p ~/jdu-lab/p1/practice01/config
cp inbox/config/training.conf practice01/config/training.conf
mv old-name.txt new-name.txt
rm unwanted.tmp
```

- `mkdir -p` は、指定したパスの途中に存在する親ディレクトリも含めて一括作成し、対象ディレクトリがすでに存在していてもエラーにしない。
- `cp SOURCE DESTINATION` はファイルやディレクトリを複製する。コピー元（SOURCE）とコピー先（DESTINATION）の指定順序を取り違えないよう注意する。
- `mv` はファイルの移動や名前の変更（リネーム）に使用する。
- `rm` はファイルを削除する。通常のGUIのような「ごみ箱」を経由せず即座に削除されるため、実行前に対象パスを `pwd` や `ls` で確認する習慣をつける。

初学者の演習では、指定ディレクトリ以下を警告なしで一括削除できる危険な `rm -rf` は使用しない。たとえば不要な `.tmp` ファイルを削除する場合も、まずは `find` コマンドで一覧を確認する。

```
find ~/jdu-lab/p1/practice01 -type f -name '*.tmp' -print
```

シングルクォートで囲まれた `'*.tmp'` は、シェルによるワイルドカードの事前展開を防ぎ、検索パターン文字列をそのまま `find` コマンドへ渡すための指定である。対象ファイルを目視で確認した上で、1件ずつ削除するか、演習で指示された安全な範囲だけを慎重に処理する。

4.6 そのほかの確認コマンド

```
ls -la
ls -ld /srv/jdu-share
tree ~/jdu-lab/m1/case01
stat -c '%U:%G %a %n' /srv/jdu-share/README.txt
```

- `ls -la` は、ドット `.` で始まる隠しファイルを含め、カレントディレクトリ内の全項目を詳細一覧で表示する。
- `ls -ld DIR` は、ディレクトリ内のファイル一覧ではなく、指定したディレクトリ自身のメタデータ（属性）を表示する。
- `tree` はディレクトリの階層構造を視覚的なツリー形式で表示する。本演習環境では初期セットアップ時にあらかじめ導入されている。
- `stat -c` は必要な項目だけを指定したフォーマットで抽出表示できる。`%U` は所有者ユーザー名、`%G` は所有グループ名、`%a` は8進数表現のアクセス権モード、`%n` はファイル名・パスである。

単一のコマンドですべてを把握しようとするのではなく、「階層構造を見たいのか（`tree`）」「中身の一覧を見たいのか（`ls`）」「属性・メタデータを詳しく見たいのか（`stat`）」という自身の目的に合わせて適切なコマンドを選択する。

4.7 nanoで編集する

`nano FILE` は、ターミナル画面内でテキストファイルを編集するための簡易エディタである。本書の演習では、編集作業を行う前に対象ディレクトリへ移動し、`pwd` と `ls` で現在地と対象ファイルを確実に確認してから起動する。

```
cd ~/jdu-lab/m0
pwd
nano observation.env
```

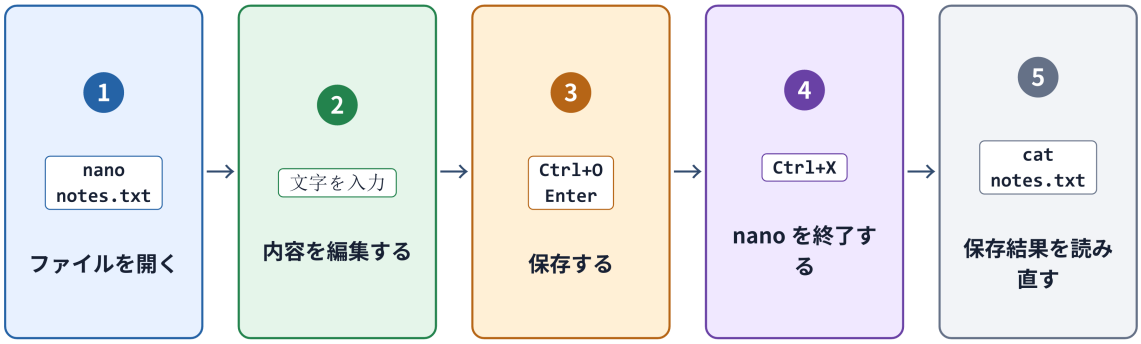


図4-2 nanoの基本操作の流れ。画面下に表示される ^ 記号は Ctrlキー を表す。

1. カーソルキー（矢印キー）で編集位置を移動し、通常のキーボード入力でテキストを編集する。
2. Ctrl+O を押してファイルの書き込み（保存準備）を開始する。
3. 画面最下行にファイル名が表示されたら、意図したパスであることを確認して Enter キーを押して保存を確定する。
4. Ctrl+X を押して nano エディタを終了する。
5. cat observation.env などで編集結果を再度読み出し、正しく保存されたことを確認する。

未保存の変更が残っている状態で終了しようとすると、変更を保存するかどうかの確認メッセージが表示される。英語表記の画面では、Y（Yes）が保存、N（No）が破棄、Ctrl+C が取り消し（キャンセル）を意味する。画面に表示された英語のメッセージを読まずにキーを乱打してはならない。

4.8 親ディレクトリのアクセス権も関係する

ファイルを読むとき、OSはパスを先頭から順にたどる。その途中にある各ディレクトリで、操作中のユーザーに通り抜ける権限があるかを確かめる。どこか一つでも通過できないディレクトリがあれば、対象ファイル自身に読み取り権限 r があっても、そのファイルを読むことはできない。namei -l コマンドは、指定したパスを構成する親ディレクトリと対象ファイルを一段ずつ表示し、それぞれの所有者とアクセス権を確認できる。

```
namei -l /srv/jdu-web/index.txt
```

ディレクトリに対する実行権限 x は、プログラムファイルの「実行」とは異なり、「そのディレクトリを経由して配下の名前にアクセスする（通り抜ける）」権限を意味する。この仕組みの詳細は第7章で詳しく解説する。

章末問題

問1 ルートディレクトリとホームディレクトリは何が違うか。

問2 `ls -l DIR` と `ls -ld DIR` では、何を表示するか。

問3 ファイル自身に読み取り権限 `r` があるのに読めない場合、`namei -l` の何を確認するか。

章末解答

問1

答え: ルートディレクトリ `/` はシステム全体のディレクトリツリーの出発点である。ホームディレクトリはユーザーごとの作業場所であり、たとえば `ssm-user` なら `/home/ssm-user` である。`~` は現在のユーザーのホームディレクトリを表す。

問2

答え: `ls -l DIR` は `DIR` の中にある項目を詳細に表示する。`ls -ld DIR` は `DIR` というディレクトリ自身の情報を表示する。

問3

答え: パスの先頭から対象ファイルまでの各親ディレクトリについて、操作中のユーザーが通り抜ける権限 `x` を持つかを確認する。途中の一つでも通過できなければ、ファイルに `r` があっても読めない。

参考資料

- Linux man-pages, path_resolution(7)（2026-09-18確認）
- Ubuntu 24.04実機の`man hier`、`man find`、`man stat`、nanoの画面（制作時に実機確認する）

第5章 テキストと入出力を組み合わせる

5.1 なぜ設定とログはテキストなのか

UbuntuをはじめとするLinuxでは、設定やログの多くがプレーンテキスト（文字列データ）として保存される。専用のGUIツールがなくても内容の確認・比較・検索が容易であり、別のコマンドへ柔軟に処理を引き渡せるためである。ただし、すべての設定やログが単純なテキストファイルであるとは限らない。systemd journalのように、バイナリ形式で記録され専用のコマンドを通じて閲覧する仕組みもある。

5.2 標準入力・標準出力・標準エラー出力

各プロセスには、標準入力（stdin: standard input）、標準出力（stdout: standard output）、標準エラー出力（stderr: standard error）という3つの基本的なデータ入出力の窓口（ストリーム）が用意されている。

- stdin（標準入力）: 通常の入力データ。キーボード入力や前段のコマンドの出力からデータを受け取る（ファイルディスクリプタ 0）。
- stdout（標準出力）: 通常の処理結果。ターミナル画面への表示や、次のコマンド、ファイルへ渡す（ファイルディスクリプタ 1）。
- stderr（標準エラー出力）: エラーメッセージや警告。通常の処理結果（stdout）と明確に分離して扱われる（ファイルディスクリプタ 2）。

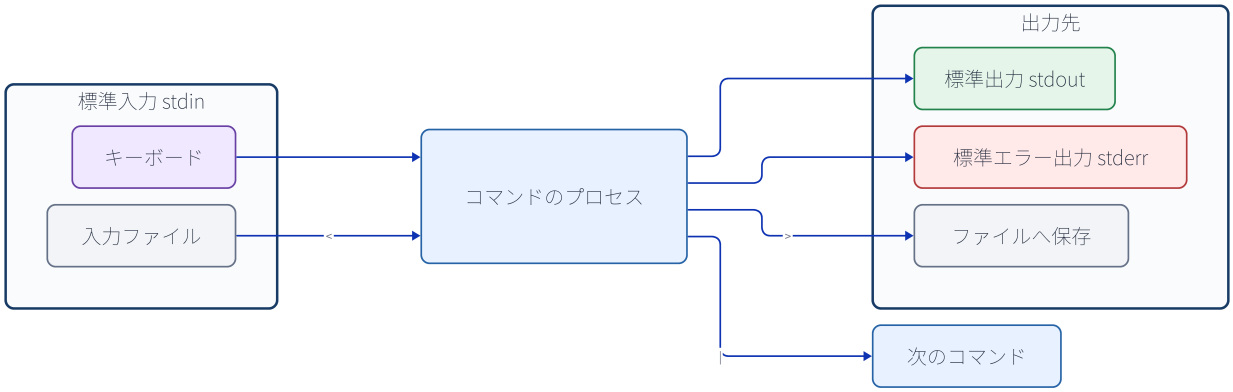


図5-1 プログラムの三つの標準ストリーム。`>` は通常stdoutをファイルへ接続し、`|` は左側のコマンドのstdoutを右側のコマンドのstdinへ接続する。

`cat incident.log`をそのまま実行すると、ファイルが入力元となり、ターミナル画面がstdoutの表示先となる。ファイルそのものがターミナルへ飛んでいくのではなく、`cat`プロセスがファイルの内容を読み出し、その文字列データをstdoutへ書き出しているのである。

5.3 `grep`で必要な行を選ぶ

```
grep 'ERROR' incident.log
```

`grep`は入力を1行ずつ走査し、指定したパターンに一致する行全体をstdoutへ出力する。

```
grep -F 'ERROR' incident.log
```

-Fは検索パターンを正規表現ではなく固定文字列（プレーンテキスト）として扱うオプションである。ERRORのような単純な英単語であれば結果は同じになることが多いが、記号文字をエスケープなしでそのまま検索したい場合に有効である。

grep -nは一致した行番号も先頭に付加して表示する。元の行だけを別のファイルへ保存したい場合は、行番号を加える-nを付けない。

終了ステータスにも重要な意味がある。grepでは一般に「パターンに一致する行があった場合」は0、「一致する行が1行もなかった場合」は1、「ファイルが存在しないなどの実行時エラー」は2となる。画面に何も出力されなかった際、単に対象文字列が存在しなかったのか、それともファイル名を間違えてエラーになっていたのかを区別するために終了ステータスを確認する。

5.4 tailで末尾を読む

```
tail -n 5 incident.log
```

tail -n 5は、ファイルの末尾5行を元の順序のままstdoutへ出力する。ここでの-nは出力行数を指定するオプションであり、grep -nの行番号付加とは全く意味が異なる。オプションのアルファベット1文字だけを丸暗記するのではなく、コマンドごとのマニュアルで仕様を確認することが重要である。

tail -fは、ファイルへ新たに追加された行を継続して表示する。表示を止めるにはCtrl+Cを押す。

5.5 リダイレクトで結果をファイルへ保存する

```
grep 'ERROR' incident.log > errors.txt
tail -n 5 incident.log > recent.txt
```

>（リダイレクト）は、シェルがstdoutの出力先をファイルへと切り替える機能である。右辺に指定したファイルがすでに存在する場合、コマンドが実行される前にファイルサイズをゼロに切り詰めて（空にして）上書きする。そのため、入力ファイルと出力ファイルに同じファイルを指定すると、コマンドがデータを読み込む前にファイルの内容が消失してしまう危険がある。

```
# 危険な例。実行しない
grep 'ERROR' incident.log > incident.log
```

>>はファイルの末尾にデータを追記する。複数回実行すると同じ行が重複して増える。毎回同じ内容のファイルを作り直すなら、>で上書きする。

標準エラー出力（stderr）は、通常>だけでは同じファイルには書き込まれない。2>を使うと、stderrの出力先を別のファイルへ指定できる。

5.6 パイプはファイルを作らず次へ渡す

```
grep 'ERROR' incident.log | tail -n 5
```

|（パイプ）は、左側のコマンドのstdoutを右側のコマンドのstdinへ直接接続する。ディスク上へ中間ファイルを作成することなく、処理結果をパイプラインとして絞り込むことができる。ただし、左側のコマンドが出力したstderrは通常パイプラインには渡されない。

パイプライン全体の終了ステータスは、Bashの標準設定では「最後に実行されたコマンド」の終了ステータスとなる。そのため、左側のコマンドが失敗していても右側のコマンドが成功すればパイプ全体として成功と判定されてしまう場合がある。厳密な自動処理スクリプトではset -o pipefailなどの利用を検討する。

5.7 クォート、変数、コマンド置換

シェルはコマンドを実行する前に、入力された文字列の展開処理を行う。

```
name='incident.log'
printf '%s\n' "$name"
```

- シングルクォート '...': 囲まれた内部の文字列をほぼ文字どおり（リテラル）に扱う。$HOMEなどの変数も展開されない。
- ダブルクォート "...": 空白を含む文字列を単一の引数としてまとめつつ、$HOMEなどの変数や$(...)などのコマンド置換は展開する。
- クォートなし: 空白による意図しない引数分割やワイルドカード展開が発生するため、変数を使用する際は原則として"$name"のようにダブルクォートで囲む。

$(command)（コマンド置換）は、指定したコマンドを実行し、そのstdout出力をその位置へ文字列として展開・代入する。

```
current_user="$(id -un)"
printf 'USER=%s\n' "$current_user"
```

5.8 &&は前が成功したときだけ続ける

```
cd /var/log && pwd
```

&&の右辺のコマンドは、左辺のコマンドの終了ステータスが0（成功）であったときのみ実行される。ディレクトリ移動に失敗したまま意図しない場所でファイルを編集・削除してしまう事故を防止できる。ただし、1行の中に何段階も詰め込むと、どの段階でエラーが発生したのか把握しづらくなる。

章末問題

問1 grep ERROR incident.log > errors.txtで、errors.txtを開くのはgrepかシェルか。

問2 >と>>の動作はどう違うか。取り違えると何が起こるか。

章末解答

問1

答え: シェルである。シェルがerrors.txtを開いてgrepの標準出力先へ接続してから、grepを実行する。

問2

答え: >はファイルを上書きし、>>は末尾へ追記する。>を誤って使うと既存の内容が失われる。>>を誤って繰り返すと同じ内容が重複する。

参考資料

- Ubuntu 24.04実機のman grep、man tail、man bash（制作時にversion確認）
- GNU Bash Reference Manual（2026-09-18はWeb本文取得timeout。実機manualを一次確認に使う）

第6章 ユーザー、グループ、ログイン

6.1 ユーザーの種類と役割

Linuxには、人が操作するためのアカウントだけでなく、OSやサービスのプロセスを動かすためのアカウントもある。LinuxはユーザーをUID（ユーザーID）、グループをGID（グループID）という数値で内部的に識別する。まず、用途と権限の違いを整理する。

種類	主な役割	対話ログインとの関係
管理者のroot	システム全体を管理する特別なユーザー。UIDは0	アカウントは存在するが、Ubuntuでは通常、rootのパスワードによる直接ログインを無効にする。必要な管理操作は許可されたユーザーがsudo（管理者などの権限でコマンドを実行する仕組み）で行う
一般ユーザー	人が自分の作業を行う。ssm-userやjduopsなど	設定と認証方法が許せば対話ログインできる。一般ユーザーであることとsudoを使えることは別である
サービス用ユーザー	Webサーバーなどのプロセスを、必要な範囲の権限で動かす。jduwebなど	対話ログインを必要としないことが多い。ログインシェルを/usr/sbin/nologinに設定するのが一般的だが、すべてのサービス用ユーザーに必須の性質ではない

この表はLinuxが内部でユーザーを三種類に固定しているという意味ではない。用途を理解するための分類である。実際にできる操作は、そのアカウントの設定、所属グループ、各ファイルの権限、sudoやSSHの設定によって決まる。また、rootのアカウントが存在することと、rootとして直接ログインできることも別である。

ssm-userのような名前は人間が扱いやすい識別名である。ファイルやディレクトリの所有者情報も、内部では数値のUID/GIDとして記録されている。

```
id
id jduops
getent passwd root
getent passwd jduops
getent passwd jduweb
getent group ops
```

idは、現在操作しているユーザーのUID、主グループ、補助グループを表示する。引数にユーザー名を指定すると、そのアカウントの登録情報を確認できる。getent passwd USERには、ユーザー名、UID、主グループのGID、ホームディレクトリ、ログインシェルなどが表示される。たとえば最後の項目が/usr/sbin/nologinなら、通常の対話シェルとしては使えない。getentはローカルファイルだけでなく、システムが設定したネームサービスからも情報を取得する。企業の環境では、LDAPやActive Directoryなどのアカウント情報が含まれる場合もある。

6.2 主グループと補助グループ

各ユーザーには、必ず1つの主グループ（primary group）が設定される。Ubuntuで新規ユーザーを作成すると、ユーザー名と同じ名前の専用グループを主グループとして自動作成する運用が一般的であるが、これは必須の絶対規則ではない。必要に応じて usermod -g コマンドなどで変更することも可能である。

1人のユーザーは、主グループに加えて複数の補助グループ（supplementary group）に所属できる。チームで共有するファイルやディレクトリへのアクセス権を付与する際は、ユーザーの主グループを不用意に変更するのではなく、用途ごとに作成した補助グループへユーザーを追加する運用が適切である。

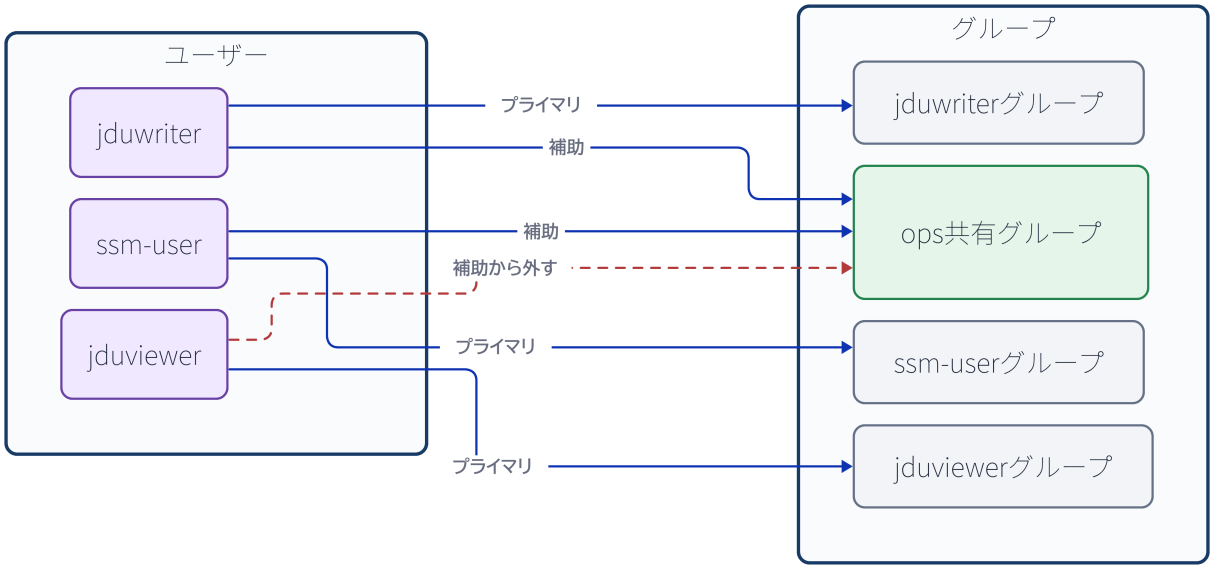


図6-1 ユーザーごとの主グループと、共有用途で利用する補助グループの関係。

補助グループへの追加と、補助グループからの削除には、次のコマンドを使う。

```
sudo usermod -aG ops jduops
sudo gpasswd -d jduviewer ops
```

`usermod -aG` において、`-a`（append）は既存の所属グループを維持したまま追加することを意味する。もし `-a` を付けずに `-G` だけでグループ名を指定してしまうと、そこに記載しなかった既存の補助グループからすべて除外されてしまう危険がある。設定を変更した後は、必ず `id USER` で登録情報を再確認する。

`getent group ops` の出力末尾にあるメンバー一覧は、そのグループを補助グループとして所属しているメンバーを確認するのに役立つ。しかし、そのグループを主グループとして登録されているユーザーは、この一覧に名前が表示されない場合がある。グループの全所属者を確実に確認する際は、`id USER` などと組み合わせて多角的に照合する。

6.3 登録情報と現在のプロセスの資格情報

グループの所属登録を変更しても、すでに起動して動作しているシェルのプロセスの資格情報（保持しているグループ一覧）が自動的に更新されるわけではない。新しくログインし直すか、新しいセッションを開始して初めて、最新のグループ情報が反映される。

```
id jduops
sudo su - jduops
id
```

1行目の `id jduops` はシステムに登録されているアカウント情報を調べる。一方、`su -` で切り替えた後の `id` は、新たに起動したログインシェルが実際に保持している実行資格情報を表す。カーネルによるアクセス権の判定は、設定ファイルの静的な登録情報ではなく、操作を実行しているプロセス自身が保持するUID/GIDに基づいて行われる。

6.4 `su`で別のユーザーのシェルを開く

`su`は、別のユーザーの権限でコマンドを実行するためのコマンドである。ここでは、別ユーザーとして操作を続けるために、そのユーザーのシェルを開く使い方を学ぶ。

```
su - jduops
id -un
pwd
exit
id -un
```

`su - jduops`は、`jduops`の権限でログインシェルを開始する。`-`は、対象ユーザーがログインしたときに近い環境を作る指定であり、ホームディレクトリへ移動し、ホームやシェルなどの環境変数を初期化する。切り替え後は`id -un`でユーザー名、`pwd`で作業ディレクトリを確認する。

元のユーザーの登録情報を書き換える操作ではない。元のシェルの内側に別ユーザーのシェルを開くため、`exit`でそのシェルを終了すると一つ前のシェルへ戻る。最後の`id -un`で元のユーザーに戻ったことを確かめる。

一般ユーザーが直接`su - USER`を使う場合、通常は対象ユーザーのパスワードを求められる。上の例は対象ユーザーのパスワード認証が可能な環境向けである。授業のUbuntuでは、次節で説明する`sudo su - USER`を用いて切り替える。パスワードが未設定のユーザーに対して、直接の`su`が成功するとは限らない。

`su`はsubstitute userと説明される。substituteは「代わりにする・置き換える」という意味であり、ここではコマンドを実行するユーザーを別のユーザーに置き換える、と理解するとよい。「ユーザーを切り替える」という意味のswitch userは、動作を覚えるための言い換えである。

6.5 sudoで別のユーザーの権限を使ってコマンドを実行する

`sudo`は、設定で許可されたユーザーが、管理者（`root`）や別のユーザーの権限でコマンドを実行する仕組みである。一般ユーザーなら誰でも使えるわけではない。誰が、誰の権限で、どのコマンドを実行できるかは、`sudoers`などの設定で決まる。管理者権限を与える対象と操作の範囲は、必要最小限にする。

対象ユーザーを指定しない通常の設定では、`root`の権限で実行する。次の例では、`sudo`を付けた`id -un`だけが`root`として動作する。その後の`id -un`は元のシェルで動作するため、元のユーザー名を表示する。

```
sudo id -un
id -un
```

パスワード認証が必要な通常の設定では、実行した本人のパスワードを求められる。対象ユーザーのパスワードを求める`su`との違いである。ただし認証方法は設定によって異なり、パスワードを求めない設定もある。

1つのコマンドを別ユーザーの権限で試す場合は、`sudo -u USER -- COMMAND`と実行する。`-u USER`は対象ユーザーの指定、`--`は`sudo`のオプションの終わりを示す。この使い方では対話シェルを開かず、指定したコマンドが終了すると元のシェルで操作を続けられる。

```
sudo -u jduweb -- test -r /srv/jdu-web/index.txt
echo $?
```

これは`jduweb`の実行資格情報で対象ファイルが読み取り可能かどうかを直接テストしている。`test -r`自体は成功しても何も表示しない。直後の`echo $?`が`0`なら読み取り可能、`1`なら読み取り不可である。間に別のコマンドを実行すると、そのコマンドの終了ステータスに変わる。`root`で読み取れたことは、サービス用ユーザーでも読み取れる証拠にはならない。

別ユーザーとして複数の操作を続ける場合、授業では次の組合せを使う。

```
sudo su - jduops
id
pwd
exit
id -un
```

これは一つのコマンド名ではなく、sudoでsu - jduopsを実行する組合せである。通常の設定では、sudoが許可を確認してrootとしてsuを実行し、suがjduopsのログインシェルを開く。exitで戻り、id -unで元のユーザーを確認する。

sudoはsuperuser do、つまり「管理者として実行する」と説明される。suのsubstitute userと合わせて英語の意味を覚えると、使い方を整理しやすい。ただし現在のsudoは、-uでroot以外のユーザーも指定できる。名前は似ているが、suとsudoは別のコマンドである。別ユーザーのシェルを開いて操作したいのか、許可されたコマンドを別ユーザーの権限で実行したいのかを区別して使い分ける。

6.6 サービス用ユーザーでプロセスの権限を分ける

サービス用ユーザーは、サービスのプロセスに与える権限を、人の作業用アカウントや他のサービスから分離するために使う。たとえばWebサーバーのプロセスをjduwebとして動かせば、アクセス権の判定にはjduwebのUIDとGIDが使われる。サービスが必要とするファイルだけを読めるように設定できる。

```
getent passwd jduweb
id jduweb
```

/usr/sbin/nologinは、通常の対話ログインに使うシェルの代わりに起動し、ログインを断って終了するプログラムである。この設定でも、アカウント自体は存在する。systemdはサービスの設定で指定したユーザーの権限でプロセスを起動できる。人がそのアカウントで対話ログインできないことと、サービスのプロセスがそのUIDでファイルを扱えることは別である。

6.7 アカウント、認証、認可を区別する

アカウント（account）は、システムに登録されたユーザーの情報である。ユーザー名とUID、主グループ、ホームディレクトリ、ログインシェルなどを結び付ける。「ssm-userというアカウントがある」とは、サーバーがその名前を識別できるという意味であり、接続してきた人が本当にその利用者かどうかまでは証明しない。

認証（authentication）は、接続してきた相手が、申告したアカウントを使用する正当な相手か確かめることである。パスワード認証なら秘密のパスワードを、SSHの公開鍵認証なら対応する秘密鍵を所持していることを確かめる。公開鍵認証では、接続先の対象ユーザーの~/.ssh/authorized_keysなどに公開鍵を登録する。秘密鍵そのものをサーバーへ送って照合するわけではない。

認可（authorization）は、その主体が特定の操作をしてよいか判断することである。SSHでは、アカウントやサーバーの設定によって接続を許すかも判断する。接続後は、ファイルを読めるか、書けるか、コマンドを管理者権限で実行できるかを、それぞれの規則で判断する。ログインに成功しても、すべての操作が許されるわけではない。

SSHでssm-userとして接続する例を、三つの概念に分けてみる。

1. アカウント: 接続先にssm-userの登録があり、UIDやホームディレクトリなどを取得できる。登録がなければ、そのユーザーとして通常のセッションを開始できない。
2. 認証: 接続元は秘密鍵を使い、接続先はssm-userに登録された公開鍵と対応することを確認する。鍵が合わなければ、アカウントが存在しても認証は失敗する。

3. 認可: 接続を許すSSHの設定を通過した後も、開始されたプロセスはssm-userのUIDと所属グループに応じて操作が制限される。たとえば別ユーザーだけが読めるファイルは読めない。sudoを使えるかも別の設定で判断される。

これは概念を理解するための整理であり、SSHプログラム内部の厳密な処理順を示すものではない。SSH鍵が正しくても、SSH側のアクセス制限やアカウントの設定によって対話シェルを使えない場合がある。また、サービス用ユーザーはSSHで認証してログインしなくても、systemdから指定されたUIDでプロセスを起動できる。アカウントを持つこと、ログインを認められること、プロセスが何かを操作できることは、それぞれ別の判断である。

章末問題

問1 主グループと補助グループはどう違うか。共有ディレクトリの例で説明せよ。

問2 グループ登録を変更しても、開いたままのシェルに反映されないのはなぜか。何を開始し直すか。

問3 sudo -u jduweb -- test -r FILEを、rootによるcat FILEで代用できないのはなぜか。

章末解答

問1

答え: 主グループはユーザーに必ず一つ設定される基本のグループである。補助グループは追加で所属できるグループである。複数のユーザーが共有ディレクトリを利用するときは、共通の補助グループを作って対象ユーザーを追加できる。

問2

答え: 動作中のシェルは、開始時に得たUIDとグループの資格情報を保持しているためである。新しくログインするか、新しいセッションを開始して、最新のグループ情報を持つシェルを起動する。

問3

答え: rootが読めても、`jduweb`のプロセスに読み取り権限があるとは限らないためである。`sudo -u jduweb`は`jduweb`の権限で読み取り可否を判定する。

参考資料

- Ubuntu Server documentation, User management（2026-09-24確認。rootと一般・システム用ユーザー、アカウント管理）
- Ubuntu Server documentation, OpenSSH server（2026-09-24確認。公開鍵認証と`authorized_keys`）
- Ubuntu 24.04 man pages, adduser(8)、systemd.exec(5)（2026-09-24確認。サービス用ユーザーとプロセスの実行ユーザー）
- Linux man-pages, passwd(5)、credentials(7)（2026-09-24確認。UID/GID、ログインシェル、プロセスの資格情報）
- Ubuntu 24.04実機の`man id`、`man getent`、`man usermod`、`man gpasswd`、`man su`、`man sudo`（制作時に確認）
- util-linux公式 su(1)（2026-10-13確認。substitute user、ログインシェル、認証）
- Ubuntu Server公式 Welcome to the terminal（2026-10-13確認。sudoをsuperuser doと説明）
- Sudo公式 sudoマニュアル（2026-10-13確認。許可、対象ユーザー、パスワードと設定）

第7章 アクセス権と共有ディレクトリ

7.1 アクセス権は誰に何を許すかを表す

基本的なLinuxのアクセス権（パーミッション）は、所有ユーザー（u: owner user）、所有グループ（g: owner group）、その他（o: others）という3つのクラス（区分）に対して、読み取り（r: read）、書き込み（w: write）、実行（x: execute）の各権限を設定する。

```
-rw-rw-r-- 1 root ops ... README.txt
drwxrwsr-x ... root ops ... /srv/jdu-share
```

先頭の1文字はファイル種別を示し、`-` は通常のファイル、`d` はディレクトリを表す。それに続く9文字を、所有者、グループ、その他の3文字ずつに分割して読み取る。

カーネルは、操作を試みるプロセスの実行資格情報（UID/GID）と、対象ファイルのメタデータを照合してアクセスの可否を判断する。ここで重要なのは、所有者・グループ・その他の権限をすべて足し合わせて判定するのではない、という点である。プロセスがファイルの所有者であれば「所有者（owner）欄」だけが評価され、所有者ではないがグループに該当すれば「グループ（group）欄」だけが評価され、どちらにも該当しなければ「その他（others）欄」が適用される。

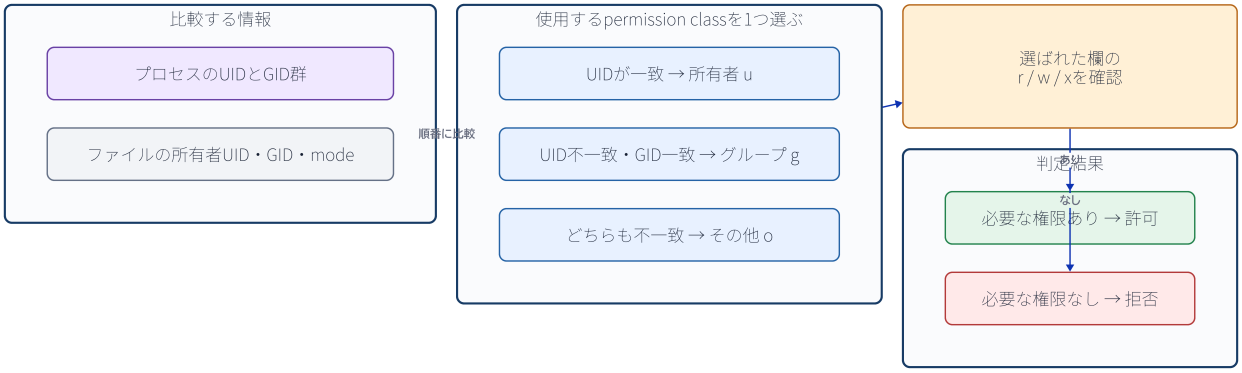


図7-1 プロセスの資格情報から u/g/o のどのクラスを適用するかを選択し、その欄の rwx で操作の可否を判断する流れ。

7.2 ファイルとディレクトリでrwxの意味が違う

権限	通常ファイル	ディレクトリ
r	内容を読み取る	内部の名前（ファイル名・ディレクトリ名）一覧を読み取る
w	内容を変更・追記する	内部に名前を新規作成・削除する
x	プログラムとして実行する	内部の名前をたどる、対象のパスへ到達する（トラバース）

ディレクトリに対して書き込み権限（w）があれば、ファイル自体に書き込み権限がなくても、そのディレクトリからファイルを削除できる場合がある（ファイル削除はディレクトリの内容変更であるため）。逆に、ファイル自体に読み取り権限（r）があっても、親ディレクトリに実行権限（x）が付与されていなければ、そのファイルへ到達できず読み取ることができない。このような多段階の権限関係を把握するには、`namei -l PATH` コマンドで経路上の全ディレクトリのアクセス権を分解して確認する。

7.3 8進数モードを読む

アクセス権の数値表記（8進数モード）は、r=4、w=2、x=1 を加算して表現する。

- 6 = 4+2 = rw-
- 7 = 4+2+1 = rwx
- 5 = 4+1 = r-x

したがって、通常ファイルの 664 は rw-rw-r--、ディレクトリの 775 は rwxrwxr-x となる。

```
sudo chmod 664 /srv/jdu-share/README.txt
sudo chmod 2775 /srv/jdu-share
```

全権限を意味する 777 は、「その他（others）」に対しても読み取り・書き込み・実行を許可する。共有グループのメンバーだけに操作を許す設計には適さない。

7.4 chownとchmod

chown は所有者ユーザーおよび所有グループを変更し、chmod はアクセス権モードを変更する。

```
sudo chown root:ops /srv/jdu-share
sudo chown root:ops /srv/jdu-share/README.txt
sudo chmod 2775 /srv/jdu-share
sudo chmod 664 /srv/jdu-share/README.txt
```

ディレクトリの所有者やアクセス権を変更しても、その配下にすでに存在する既存ファイルやサブディレクトリの属性が自動的に一括変更されるわけではない。-R オプションによって再帰的に配下全体を変更することも可能であるが、想定外のファイルまで意図せず書き換えてしまう危険を伴う。

```
stat -c '%U:%G %a %n' /srv/jdu-share /srv/jdu-share/README.txt
```

7.5 setgidディレクトリが解決する問題

通常、新規に作成されたファイルの所有グループは、作成したプロセスの主グループ（primary group）がそのまま適用される。そのため、複数のユーザーが単一の共有ディレクトリ内で作業を行うと、作成者ごとにファイルのグループがばらばらになり、他のメンバーが編集できなくなる問題が生じ得る。

ディレクトリに setgid ビット（Set Group ID）を設定しておくと、そのディレクトリ配下で新規作成されるファイルやサブディレクトリが、作成者の主グループではなく、親ディレクトリの所有グループを自動的に引き継ぐようになる。たとえば共有先 /srv/jdu-share が root:ops 2775 なら、新規ファイルの所有グループは通常 ops になる。

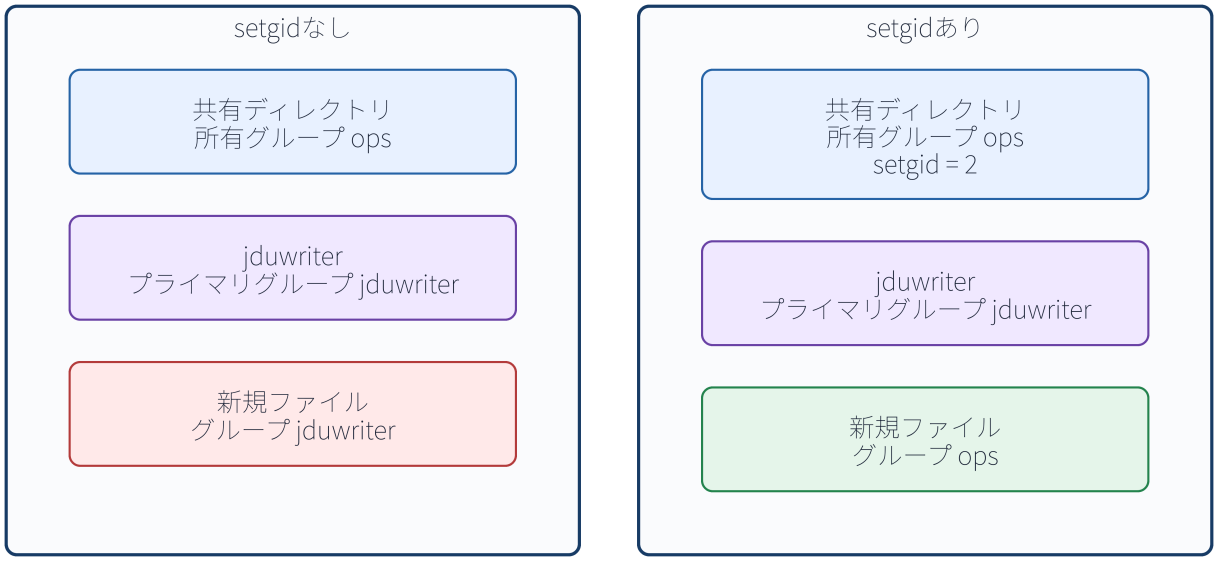


図7-2 setgid は新規項目の所有グループを親ディレクトリとそろえる。所有者、ファイル内容、アクセス権モード全体をコピーする機能ではない。

8進数モード表記の先頭にある数値 `2` が setgid ビットを表す。記号（シンボリック）表記では、グループの実行権限の位置が `x` ではなく `s` と表示される。

```
drwxrwsr-x root ops /srv/jdu-share
```

グループに実行権限（x）が付与されていない状態で setgid だけが設定されていると、大文字の `S` と表示される。共有ディレクトリでは通常グループに `rwx`（実行権限を含む）を与えるため、小文字の `s` として表示される。

7.6 umaskは新規モードの初期値へ影響する

setgid は所有グループの継承を制御する機能であり、新規作成されるファイルのアクセス権モード全体を `664` に固定するわけではない。新規ファイル作成時の実際のアクセス権は、アプリケーションプログラムが要求する基本モード値から、umask（ユーザーマスク）によって制限・マスクされたビットを差し引くことで決定される。

```
umask
touch /srv/jdu-share/sample.txt
stat -c '%U:%G %a %n' /srv/jdu-share/sample.txt
```

たとえば、通常ファイルが基本値 `666` を基準に作成される際、umask が `002` であれば、結果として `664`（`rw-rw-r--`）が設定される。ただし、プログラムが最初から別のモード値を要求することもある。なお、umask は新規作成時にのみ作用し、すでに存在するファイルのアクセス権を後から自動変更することはない。

7.7 許可と拒否を実際のユーザーで試す

メタデータの表示を目視するだけでなく、実際に対象ユーザーの権限で操作を試行して確認する。

```
sudo -u jduviewer -- test -r /srv/jdu-share/README.txt
echo $?
sudo -u jduviewer -- test -w /srv/jdu-share
echo $?
```

終了ステータス `0` はテスト条件が成立（成功）したことを示し、`1` は不成立（失敗）を意味する。ディレクトリ内にファイルを新規作成できるかどうかは、そのディレクトリ自身の書き込み権限（w）と実行権限（x）に依存する。既存ファイルの書き込み可否だけを確認して、ディレクトリへの作成権限の代用としてはならない。

アクセス拒否が起きたときは、操作したプロセスのユーザーとグループ、適用された権限クラス、不足した権限を順に確認する。単に`777`を設定して拒否を解消すると、他のユーザーにも不要な操作を許してしまう。必要なユーザーに必要な権限だけを与える。

章末問題

問1 アクセス権モード`2775`の各桁は何を表すか。

問2 setgidディレクトリ内で新規ファイルを作ると、何が親から引き継がれ、何は引き継がれないか。

問3 rootがファイルを読めても、サービス用ユーザーが読める証明にならないのはなぜか。

章末解答

問1

答え: 先頭の`2`はディレクトリのsetgidである。続く`7`は所有者の`rwx`、次の`7`は所有グループの`rwx`、最後の`5`はその他の`r-x`を表す。ディレクトリなら記号表記は`rwxrwsr-x`になる。

問2

答え: 新規ファイルの所有グループは、通常、親ディレクトリの所有グループを引き継ぐ。所有ユーザーやアクセス権モード全体はコピーされない。新規ファイルのモードには、作成プログラムが要求する値とumaskなどが影響する。

問3

答え: rootは通常のファイルアクセス権による制限を越えて読み取れる場合がある。一方、サービス用ユーザーのプロセスには、そのUIDと所属グループに応じた権限判定が適用されるためである。

参考資料

- Linux man-pages, path_resolution(7)（2026-09-18確認）
- Ubuntu 24.04実機の`man chmod`、`man chown`、`man umask`（制作時に確認）

第8章 プログラム、プロセス、PID、シグナル

8.1 プログラムとプロセスは違う

プログラムとは、ストレージ（保存領域）に保存された命令群や関連データファイルである。一方、プロセスとは、そのプログラムがメモリ上に読み込まれ、OSによって実行されている状態の存在を指す。同一のプログラムから複数のプロセスを起動することが可能であり、それぞれのプロセスには一意の識別番号であるPID（プロセスID）が割り当てられる。

なお、パッケージはプログラムや設定ファイルを配布・導入するための単位であり、サービスはバックグラウンドで継続的な機能を提供・管理するための単位である。次の関係性を混同してはならない。

```
パッケージをインストールする → プログラムファイルが配置される
プログラムを実行する　　→ プロセスが生成される
systemdがユニットを管理　→ サービスプロセスを起動・監視できる
```

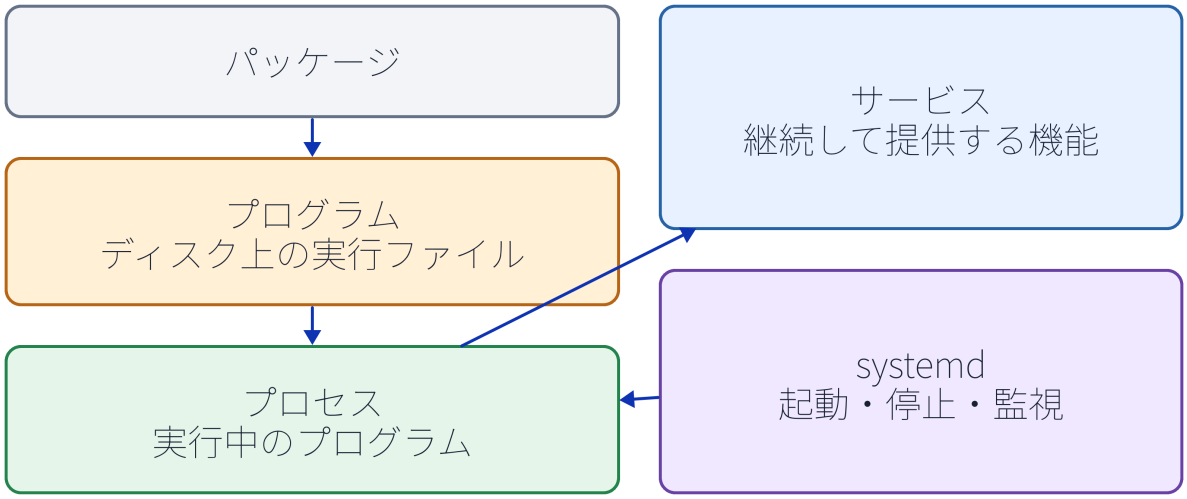


図8-1 保存されたもの（プログラム）、実行中のもの（プロセス）、管理単位（サービス）、配布単位（パッケージ）を明確に区別する。

8.2 PIDと親子関係

カーネルは、起動された各プロセスに対して正の整数値であるPID（プロセスID）を一意に割り当てる。プロセスは別のプロセスを子プロセスとして生成（起動）することができ、PPID（親プロセスID）を参照することでプロセスの親子関係を観測できる。

```
ps -e -o pid,ppid,user,stat,comm,args
ps -p 1234 -o pid,ppid,user,stat,comm,args
```

上の `1234` は表示例であり、実際のPIDは起動のたびに変わる。プロセスが終了した後は、同じ番号が別のプロセスに再利用されることもある。シグナルを送る前には、対象のPIDとともに、ユーザー名（`user`）、コマンド名（`comm`）、引数（`args`）を確認する。

PID 1 はシステムの起動初期から動作し続ける特別な管理プロセスである。Ubuntuでは通常、systemdがこの役割を担う。

8.3 サービスが管理するプロセスを見つける

Ubuntuでは、systemdがサービスの起動と状態を管理する。サービスを定義する設定をユニット（unit）と呼び、サービスのユニット名は通常 .service で終わる。設定で指定されたプログラムが実行されると、サービスに対応するプロセスが動く。ユニットという管理対象と、現在動いているプロセスは同じものではない。

systemctl はsystemdが管理するユニットの状態を調べるコマンドである。次の例では、稼働中のサービスを一覧表示し、一つのサービスの状態と主なプロセスを調べる。

```
systemctl list-units --type=service
systemctl status systemd-journald.service --no-pager
```

status の Main PID は、そのサービスの主なプロセスのPIDである。表示された番号をメモし、ps -fp PIDのPIDをその番号へ置き換えて照合すると、サービスの管理情報と実際のプロセスを結び付けられる。ps はプロセスを調べ、systemctl はサービスの管理状態を調べる。systemdが管理するサービスを停止するには、PIDへ直接シグナルを送る代わりにsystemctl stop サービス名を使う。これによりsystemdが停止処理を管理する。サービスの自動起動設定は第10章で扱う。

8.4 フォアグラウンドとバックグラウンド

ターミナルから通常のコマンドを実行すると、シェルはそのコマンドがフォアグラウンド（前景）プロセスとして終了するまで待機する。その間、キーボードからの入力は主にそのプロセスへ直接渡される。Ctrl+C は、ターミナルからフォアグラウンドのプロセスグループ全体へ割り込みシグナルを送信する操作である。

コマンドの末尾に&を付けると、シェルはそのコマンドをバックグラウンド（背景）で開始し、次の入力を受け付ける。

8.5 シグナルはプロセスへの通知

シグナル（signal）は、OSやプロセスから別のプロセスに対して、特定の出来事や制御要求を非同期に通知する仕組みである。

- TERM（SIGTERM）：プロセスに対して、データの保存やリソース解放などの正常な終了処理を行う猶予を与えて終了を要求する。プロセスを終了させる際に最初に使用すべき標準シグナルである。
- KILL（SIGKILL）：カーネルが対象プロセスを強制的に即座終了させる。プロセス側でシグナルを捕捉・後始末することはできない。TERM 等の通常シグナルを受け付けない暴走時の最終手段である。
- INT（SIGINT）：ターミナルの Ctrl+C などから送信される割り込みシグナルである。

```
kill -TERM PID
```

killコマンドは、その名称とは異なり、対象プロセスへ指定したシグナルを送るコマンドである。たとえばpkill 1956とすると、1956はPIDではなくプロセス名の検索パターンとして解釈される。PIDを直接指定する場合はkillを用い、プロセス名のパターンで指定する場合はpkillを用いる。

8.6 プロセスを終了してもファイルは消えない

プロセスを終了させても、ストレージ上に保存されているプログラムファイル、ユニットファイル、データファイルなどはそのまま残る。逆に、ディスク上の実行ファイルを削除したとしても、すでにメモリ上に読み込まれている実行中プロセスが直ちに停止するわけではない。

systemdで管理されるサービスは、プロセスが終了した後に再起動される場合がある。再起動する条件は、ユニットファイルのRestart=などの設定で決まる。したがって、プロセスが終了したことと、サービスが停止し続けることも別である。

8.7 /proc/PIDで実行状態を見る

```
printf 'PID=%s\n' "$$"
cat "/proc/$$/cmdline" | tr '\0' ' '
printf '\n'
readlink -f "/proc/$$/cwd"
```

$$は現在のBashシェルのPIDである。/proc/PID/cmdlineでは、引数群がNUL文字（\0）で区切られているため、trで空白に置き換えて表示する。/proc/PID/cwdは、そのプロセスの作業ディレクトリを指すシンボリックリンクである。プロセスが終了すると、対応する/proc/PIDも消える。

/proc/PIDを読むと、保存されたプログラムファイルではなく、現在動作しているプロセスの情報を観測できる。

章末問題

問1 パッケージ、プログラム、プロセス、サービスはどう違うか。

問2 シグナルを送る前に、PIDをpsで照合するのはなぜか。

問3 TERM送信後にサービスが再び動いたら、ユニットファイルの何を確認するか。

章末解答

問1

答え: パッケージはプログラムや設定を配布・導入する単位である。プログラムは保存された命令群である。プロセスはプログラムの実行中の状態である。サービスは継続的な機能を提供し、systemdなどが管理する単位である。

問2

答え: PIDは起動のたびに変わり、終了後に別のプロセスへ再利用されることがあるためである。PIDだけでなく、実行ユーザー、コマンド名、引数を照合し、意図したプロセスか確認する。

問3

答え: `Restart=`などの再起動設定を確認する。プロセスが終了しても、systemdがその設定に従って再起動する場合がある。

参考資料

- Linux man-pages / Ubuntu 24.04の`man ps`、`man kill`、`man proc`（制作時に実機確認）
- GNU Bash Reference Manual, Special Parameters（2026-09-24確認。`$$`はシェルのPID）
- Ubuntu 24.04 man page, systemd.service(5)（2026-09-24確認。`Restart=`）
- Ubuntu 24.04 man page, systemctl(1)（2026-09-24確認。`status`と`show`）

第9章 パッケージとソフトウェアの導入

9.1 パッケージが必要な理由

実用的なソフトウェアは、単一の実行ファイルだけで完結するとは限らない。動作に必要な共有ライブラリ、設定ファイルのテンプレート、マニュアル文書、データファイルなどを同時に必要とする。パッケージ（package）とは、これら関連するファイル群、バージョン情報、依存関係、およびシステムの導入・削除スクリプトをひとまとめにした配布単位である。

Ubuntuでは主に Debian パッケージ形式（.deb）を採用し、それを APT（Advanced Package Tool）で一括管理する。APT はリポジトリ（repository）から利用可能な最新のパッケージ情報を取得し、必要な依存ライブラリを自動で解決しながらシステムへ導入する。インターネット上から拾ってきた素性の知れないスクリプトを不用意に root 権限で直接実行することと、公式に信頼されたリポジトリから検証済みのパッケージを導入することは、セキュリティの観点から根本的に異なる行為である。

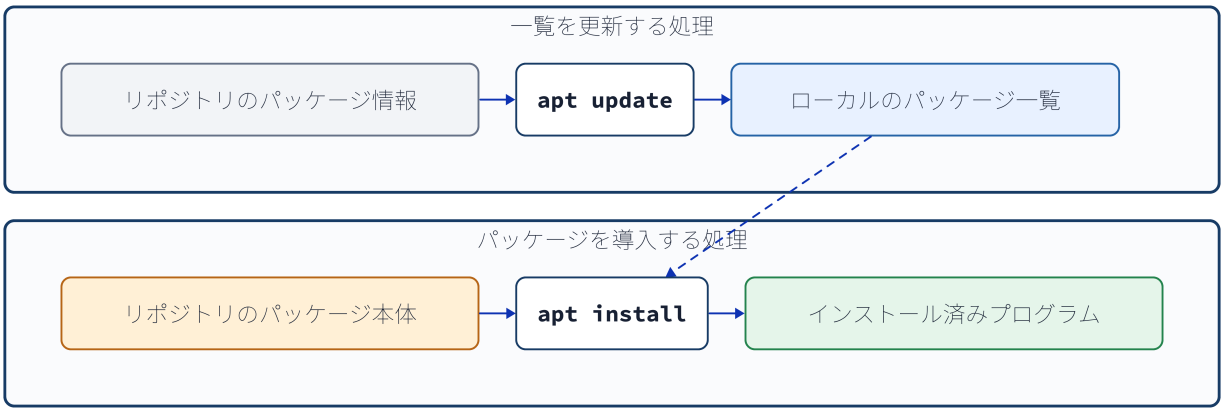


図9-1 apt update はインデックス（一覧情報）を更新し、apt install がパッケージ本体を取得・導入する流れ。

9.2 リポジトリとパッケージインデックス

リポジトリ（repository）とは、パッケージ本体とそれに付随するメタデータ（バージョンや依存関係の説明）を提供する配布拠点のサーバーである。手元のUbuntuシステムは、どのリポジトリにどのバージョンのパッケージが存在するかをローカルのパッケージインデックス（一覧情報）として保持している。

```
sudo apt update
apt show cmatrix
sudo apt upgrade
```

apt update は、リポジトリから利用可能なパッケージの一覧（インデックス）を最新状態に更新する。これだけでは導入済みパッケージの本体は変わらない。apt show cmatrix では、導入前にパッケージの説明やバージョン、依存関係を確認できる。apt upgrade は、更新した一覧を基に、導入済みパッケージを新しいバージョンへ更新する。実行時に表示される変更内容を確認してから、継続するか判断する。これはUbuntuの新しいリリースへ切り替える操作ではない。

9.3 インストールと実行確認

```
sudo apt install cmatrix
command -v cmatrix
cmatrix
```

sudo 権限が必要となるのは、システム全体のパッケージデータベースや、管理者保護下にある /usr/bin などの配置領域を変更するためである。導入が完了したら、command -v を用いてシェルがどのパスの実行ファイルを認識しているかを確認する。cmatrix は Ctrl+C で終了できる。

パッケージがシステムに導入済みであることと、現在そのプロセスがメモリ上で動作していることは全く別の状態である。コマンドを終了させてもパッケージ自体はシステムに残り続ける。その一方で、サーバー系ソフトウェアのパッケージによっては、導入完了時にサービスプロセスを自動起動する設定になっているものもある。思い込みで判断せず、systemctl コマンドを用いて実際の稼働状態を確認することが重要である。

9.4 バージョンとファイルの所属を確認する

```
dpkg-query -W -f='${Package} ${Version}\n' cmatrix
dpkg -S /usr/bin/apt
```

dpkg-query は、ローカルのパッケージデータベースから現在導入されている正確なバージョン情報を読み出す。dpkg -S PATH は、指定したファイルパスがどのパッケージによって提供・配置されたかを調べる。command -v が「シェルが実行対象として認識しているファイルパス」を答えるのに対し、dpkg -S は「そのファイルを管理しているパッケージの所属関係」を答える。/usr/bin/aptは例である。コマンドの公開パスがシンボリックリンクの場合、そのパス自体はパッケージの管理対象でないことがある。readlink -fで実体のパスを確認してから、その実体を dpkg -Sで調べる。

9.5 削除とクリーンアップ

apt remove はパッケージ本体を削除する。apt purge は設定ファイルも含めて削除する。apt autoremove は、ほかのパッケージから必要とされなくなった依存パッケージを削除する。いずれも apt upgrade とは異なり、削除されるものを確認してから実行する。

章末問題

問1

apt update、apt upgrade、apt install の役割の違いを説明せよ。

問2

command -v cmatrix と dpkg -S /usr/bin/apt がそれぞれ何を調べるコマンドであるか、その違いを説明せよ。

問3

パッケージが正常に導入されていても、対応するプロセスが動いていない状況を一例挙げよ。

章末解答

問1

答え： `apt update` は利用可能なパッケージの一覧を更新する。`apt upgrade` は導入済みパッケージを新しいバージョンへ更新する。`apt install` は指定したパッケージを導入する。

問2

答え： `command -v cmatrix` はシェルが実行するコマンドのパスを調べる。`dpkg -S /usr/bin/apt` は、そのファイルを提供するパッケージを調べる。公開パスがシンボリックリンクなら、実体のパスを確認してから所属を調べる。

問3

答え： `cmatrix` を導入したあと、起動せずにいる場合や、起動後に `Ctrl+C` で終了した場合。パッケージは残るが、`cmatrix` のプロセスは動いていない。

参考資料

- Ubuntu Server documentation, Install and manage packages（2026-09-24確認）
- Ubuntu Server documentation, Managing your software（2026-09-24確認）
- Ubuntu 24.04実機の`man apt`、`man dpkg-query`、`man dpkg`（制作時に確認）

第10章 systemdとサービス

10.1 サービスとは何か

サービス（service）とは、利用者が毎回ターミナルから手動で起動しなくても、OSシステムがバックグラウンドで自動管理し、継続的な機能を提供するプログラムである。Webサーバー、SSHサーバー、時刻同期、ログ管理デーモンなどがその代表例である。ただし、システム上で動作するすべてのプロセスがサービスであるわけではなく、またすべてのサービスがネットワーク待受用のポートを持つわけでもない。

Ubuntu 24.04では、最上位プロセスである systemd が PID 1 として起動し、システム内の多様なサービス群を一元管理する。ここで重要なのは、systemd は「個々のサービス処理そのもの」を行うのではなく、各種サービスの起動、停止、依存関係の解決、死活監視、システム起動時の自動起動などを統括管理する「サービスマネージャー」プログラムである、という点である。

10.2 ユニットファイルは設定、プロセスは実行状態

systemd はユニット（unit）という統一的な管理単位を用いて各種リソースを制御する。その中でもサービスを定義する `.service` ユニットファイルには、「どの実行ファイルを、どのユーザーの権限で、どのディレクトリを作業場所として起動するか」といった実行ルールが明確に記述されている。

```
[Service]
User=jduapp
WorkingDirectory=/srv/jdu-status
ExecStart=/usr/bin/python3 /srv/jdu-status/server.py
```

- `User`: サービスプロセスを実行するユーザー（資格情報）。
- `WorkingDirectory`: プロセス起動時のカレントディレクトリ（作業ディレクトリ）。
- `ExecStart`: 実際に起動するプログラムの絶対パスと引数。

```
systemctl cat jdu-status.service
```

`systemctl cat` コマンドは、ディスク上のユニットファイルと、設定を部分上書きするドロップイン設定（drop-in）の内容を表示する。ファイルの変更後に `daemon-reload` を実行していない場合、表示内容とsystemdが認識している設定は一致しないことがある。

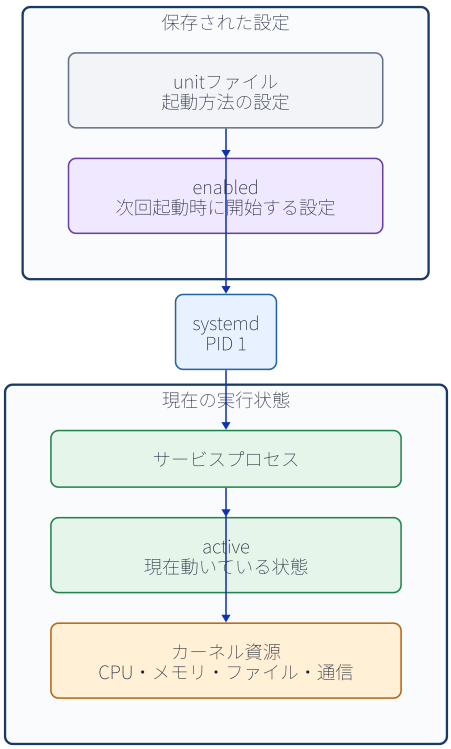


図10-1 ユニット（unit）は設定、systemd は管理者、サービスプロセスが実際の処理を行う構造。

10.3 activeとenabledは別の軸

```
systemctl is-active jdu-status.service
systemctl is-enabled jdu-status.service
```

- active / inactive: そのサービスプロセスが現在メモリ上で動作しているか、停止しているか。
- enabled / disabled: システムのブート（起動）時に、自動的に起動対象として組み込まれているか否か。

したがって、「現在は active（稼働中）だが disabled（再起動時には自動起動しない）」という状態や、「現在は inactive（停止中）だが enabled（次回起動時には自動起動する）」という状態が当然に成立し得る。サービスの即時起動（`start`）と自動起動の有効化（`enable`）は、全く別軸の独立した操作である。

```
sudo systemctl start jdu-status.service
sudo systemctl enable jdu-status.service
```

`enable --now` オプションを使うと、起動と自動起動設定を同時に行える。

10.4 start、stop、restart、reload

- start: 停止しているサービスプロセスを起動する。
- stop: 動作中のサービスプロセスを停止する。
- restart: サービスを一度停止させてから再度起動する。一時的な通信断（停止時間）が発生する。
- reload: プロセスを停止させることなく、設定ファイルの再読み込みだけを要求する。すべてのサービスがこの機能に対応しているわけではない。

なお、ユニットファイル自体の記述を変更した際に systemd 本体へ再読み込みを指示する `systemctl daemon-reload` と、サービスプロセス自身に設定を再読込させる `reload` は全く別の操作である。

10.5 ユニット設定と実プロセスを照合する

```
systemctl status jdu-status.service
systemctl show --property MainPID --value jdu-status.service
```

第8章ではMain PIDからプロセスを特定した。ここではさらに、ユニットのUser=、ExecStart=、WorkingDirectory=と実際のプロセスを照合する。systemctl statusはユニットのロード状態、稼働状態（active）、Main PID、直近の関連ログを表示する。出力行数が多い場合はページャーが起動するため、qキーで終了する。systemctl showは指定したプロパティの値だけを取得できる。

Main PIDを取得したら、その番号のプロセスとユニット設定が合致しているかを照合できる。次の1234は説明用の番号であり、実際には表示されたMain PIDへ置き換える。

```
ps -p 1234 -o pid,user,comm,args
sudo readlink -f /proc/1234/cwd
```

確認する対応は次のとおりである。

ユニットの設定	実プロセスで確認する場所
User=	ps の USER 欄
ExecStart=	ps の args 欄、および /proc/PID/cmdline
WorkingDirectory=	/proc/PID/cwd が指すシンボリックリンク先

10.6 activeだけでは機能正常を証明しない

systemd がプロセスを active（稼働中）として管理していたとしても、コンテンツファイルに対するアクセス権が不足して読めない、意図しないIPアドレスで待受（listen）している、あるいはアプリケーション内部で例外エラーが発生しているなどの理由で、利用者に正常な応答を返せない場合がある。次章では、サービス管理の状態にとどまらず、プロセス、ソケット、HTTPレスポンス、ログ出力へと確認範囲を広げていく。

10.7 systemctl statusにすべてのプロセスが並ぶわけではない

引数を指定せずに systemctl status を実行すると、システム全体の稼働サマリーやプロセスツリーが表示され、個々のユニットの詳細が必ずしも見やすく一覧化されるわけではない。目的のサービスを確認する際は、必ずユニット名を明示的に指定して実行する。

```
systemctl status systemd-journald.service --no-pager
systemctl list-units --type=service 'systemd-*'
```

ユーザーがターミナルで起動したシェルプロセスなど、systemd のサービスユニット配下に属さないプロセスも多数存在する。システム全体のプロセス一覧を網羅的に確認するには ps コマンドを使用し、systemd の管理対象であるサービスの状態を確認するには systemctl コマンドを使用する、という使い分けを明確にする。

章末問題

問1

ユニットファイル、systemd、サービスプロセスの3者の役割の違いを説明せよ。

問2

サービスの状態において「active かつ disabled」という組み合わせは成立し得るか。その状態が何を意味するかを含めて説明せよ。

問3

ユニットファイルに定義された `WorkingDirectory` の設定が、実際の稼働プロセスへ正しく反映されているかを `/proc` 配下の情報を用いて確認する方法を説明せよ。

章末解答

問1

答え： ユニットファイルは起動方法などの設定を記す。systemdはその設定を使ってサービスを管理する。サービスプロセスは、実際の処理を行う稼働中のプログラムである。

問2

答え： 成立し得る。`active`は現在動いていること、`disabled`はそのユニットの自動起動が有効化されていないことを示す。手動の`start`で起動したサービスが、その状態になり得る。

問3

答え： 稼働中サービスのMain PIDを`systemctl show --property MainPID --value ユニット名`で調べる。`readlink -f /proc/PID/cwd`で、そのプロセスの現在の作業ディレクトリを確認する。表示されたパスをユニットファイルの`WorkingDirectory=`と照合する。

参考資料

- Ubuntu 24.04 systemctl(1)、systemd.exec(5)、proc_pid_cwd(5)（2026-09-24確認）
- 公開Lab v1.0.0のP4/M4 unitとchecker（Lab固有値の正本）

第11章 ソケット、HTTP、ログでサービスを観察する

11.1 サービス、プロセス、ソケットの関係

systemd はユニットの設定定義に従ってサービスプロセスを起動する。そのプログラムがネットワーク通信を必要とする場合、システムコールを通じてカーネルへソケット（socket）の作成を依頼する。カーネルはIPアドレス、通信プロトコル、ポート番号などに基づいて、外部からの通信データを該当するプロセスへ正しく届ける。

サービスが勝手にソケットを生み出すというよりは、「サービスとして動作するプログラムが、カーネルの提供するソケット機能を利用して通信を行っている」と理解するのが正確である。systemd の役割はそのプロセスを管理・監視することであり、ネットワーク通信の送受信処理そのものをすべて systemd が代行しているわけではない。

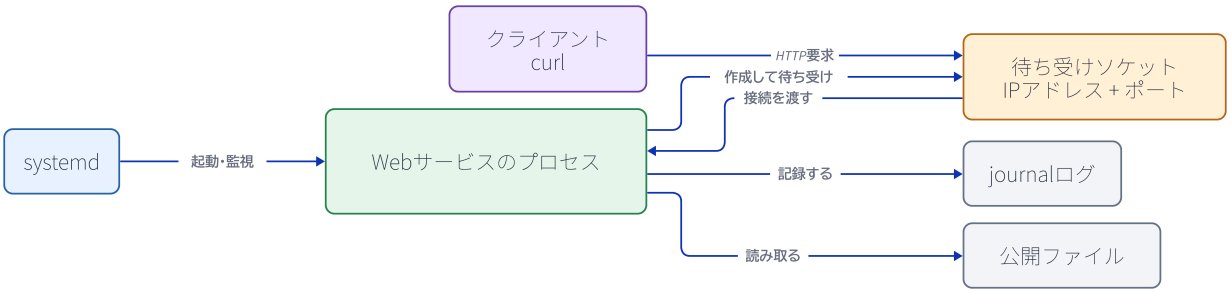


図11-1 管理（systemd）、実行（プロセス）、通信機能（カーネル・ソケット）、クライアント要求（リクエスト）の各経路の分離構造。 1つのサービスが複数のソケットを持つ場合や、ネットワーク通信を行わないサービスも存在する。

11.2 プロトコル、IPアドレス、ポート

Web通信でよく使われるトランスポート層の通信プロトコルの一つが TCP（Transmission Control Protocol） である。TCP はデータが相手へ届いたことを確認し、順序どおりに届ける通信方式である。接続の確立や再送などの詳しい仕組みは後続のネットワーク単元で扱う。

IPアドレスはネットワーク上のインターフェース（通信相手のホスト）を識別する。一方、ポート（port）番号は、同一ホスト内で動作しているどの通信窓口（どのプロセス）へデータを届けるかを識別するための番号である。「ポート番号そのものがプログラム」なのではない。カーネルがソケットの待受情報とポート番号を照合し、該当するプロセスへ通信データを引き渡す。

- `127.0.0.1`: IPv4のループバックアドレス（loopback）。外部ネットワークからは直接到達できず、基本的に同一ホスト内からの通信のみを受け付ける。
- `0.0.0.0`: そのホストが持つすべてのIPv4ローカルアドレス（すべてのネットワークインターフェース）で待ち受けることを意味する特殊表記。
- `[::]`: すべてのIPv6ローカルアドレスで待ち受ける指定。OSの設定によってはIPv4通信も同時に受け付ける場合がある。

11.3 待受ソケットを`ss`で読む

`ss`（Socket Statistics）は、カーネルが管理しているソケットの詳細な状態を表示するコマンドである。

```
sudo ss -lntp | grep ':8081'
```

- `-l`: 待受（LISTEN）状態のソケットのみを表示する。

- -n: サービス名（httpなど）へ逆引き変換せず、ポート番号の数値をそのまま表示する。
- -t: TCPソケットのみを表示する。
- -p: そのソケットを利用しているプロセスの情報（名前やPID）を表示する。表示には管理者権限が必要な場合がある。

概念上の例:

```
State  Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0      5      127.0.0.1:8081  0.0.0.0:*      users:(("python3",pid=2451,fd=3))
```

出力を読む順序は、状態（State: LISTEN）、待受アドレスとポート（Local Address:Port: 127.0.0.1:8081）、そして利用プロセス情報（Process: pid=...）である。待受アドレスが 127.0.0.1:8081 であり、プロセス情報内の PID が systemd で確認したサービスの Main PID と一致しているかを照合する。なお、2451 は教科書の表示例であり、必ず自身の環境で出力された実機の PID を読み取ること。



図11-2 ss -lntp の出力行を、状態・アドレス・ポート・プロセス情報へ分解して照合する読み方。

11.4 クライアントとサーバー、HTTP

サーバー（server）プロセスは、ソケットを作成してクライアントからの接続要求（リクエスト）を待ち受ける。クライアント（client）は、サーバーのソケットへ接続を確立し、リクエストを送信する。本演習環境では、同一のUbuntuホスト内で実行する curl コマンドがクライアントとなり、jdu-web.service のプロセスがサーバーとなる。

```
curl -i http://127.0.0.1:8081/
```

URL http://127.0.0.1:8081/ において、http は通信プロトコル（スキーマ）、127.0.0.1 は接続先ホスト、8081 はポート番号、/ はリクエスト対象のパスである。-i オプションは、応答本文（ボディ）だけでなく HTTP レスポンスヘッダーもあわせて表示する。

```
HTTP/1.0 200 OK
Content-Type: text/plain

（本文）
```

200（200 OK）は、HTTP アプリケーション層におけるステータスコードである。「TCP レベルのネットワーク接続が成功したこと」「HTTP レベルの応答ヘッダーが返ってきたこと」「応答本文（ボディ）の内容が期待どおりであること」の3点は、それぞれ異なるレイヤーの成否として個別に検証する必要がある。

11.5 オープンポートとクローズドポートの比較

```
curl --max-time 2 http://127.0.0.1:18081/
sudo ss -lnt | grep ':18081'
```

この例で 127.0.0.1:18081 に待受ソケットがなければ、接続要求は通常拒否され、curl は Connection refused と表示する。ss -lnt にそのアドレスとポートの行がないことは、そこで待ち受けるTCPソケットがないことを示す。--max-time 2 は、curl の処理全体を最長2秒に制限する指定である。

待受ソケットが存在することと、HTTPの応答が返ることも別である。待受は接続を受け付ける準備を示す。HTTPの応答には、接続後にサーバープログラムがリクエストを処理する必要がある。また、タイムアウトは通信が時間内に完了しなかったという結果であり、それだけで「待受ソケットがない」とは判断できない。

11.6 サービスユーザーの読み取りアクセス権

Webサーバープロセスが要求されたコンテンツファイルを読み出してクライアントへ返すためには、そのサービスの実行ユーザーが、コンテンツファイル自身およびその親ディレクトリ群への正当なアクセス権（読み取り権限と実行権限）を保持していなければならない。

```
id jduweb
stat -c '%U:%G %a %n' /srv/jdu-web/index.txt
namei -l /srv/jdu-web/index.txt
sudo -u jduweb -- test -r /srv/jdu-web/index.txt
echo $?
```

jduweb ユーザーへ対話的に su - ログインできないことは何ら問題ではない。サービス専用ユーザーは不正利用防止のために対話シェルが無効化されているのが標準であり、第6章で学んだ sudo -u を用いることで、対話ログインを開くことなくそのユーザーの資格情報でファイルのアクセス可否を確実にテストできる。

11.7 ログはどこから来るのか

プログラムは動作中にさまざまなログメッセージを出力する。systemd 環境のログ収集デーモンである systemd-journald は、各サービスプロセスが出力した標準出力（stdout）や標準エラー出力（stderr）、およびシステムログメッセージを一元的に収集してバイナリ形式で蓄積する。journalctl コマンドは、その蓄積されたジャーナルログを閲覧・検索するために使用する。

```
curl -i http://127.0.0.1:8081/
sudo journalctl -u jdu-web.service --no-pager -n 20
```

-u は対象ユニットでログを絞り込み、-n 20 は直近の20件を表示する。--no-pager はページャーを起動せずに結果を表示する。Webサーバーがアクセスを記録する設計なら、リクエストに対応するログ行が残る。ログの内容や有無はプログラムの実装と設定によって異なる。

11.8 Webサービスが要求に応じるまで

Webサービスを例にすると、設定・実行・通信・応答・ログは、次のようにつながる。

```
ユニットファイル → systemd → サービスプロセス
                    ↓ カーネルのソケット機能を利用
              待受ソケット（IPアドレスとポート）
                    ↓ クライアントが接続・要求
              サービスプロセスが要求を処理
                ├── HTTPレスポンスを返す
                └── 必要に応じてログを出力する
```

ユニットファイルは起動方法を定めるが、通信内容を処理するのはサービスプロセスである。待受ソケットは接続を受ける窓口であり、HTTPレスポンスはプログラムが要求を処理した結果である。ログは、その処理やエラーを後から知るための記録である。`systemctl` はサービスの状態、`ss` はソケット、`curl` はHTTP応答、`journalctl` はジャーナルに収集されたログをそれぞれ表示する。これらは同じ事実を別名で示すものではない。

章末問題

問1

サービスが active（稼働中）であるにもかかわらず、HTTPで期待した応答を得られない原因の例を2つ挙げよ。

問2

待受アドレス `127.0.0.1:8081` と `0.0.0.0:8081` の違いを、外部ネットワークからの到達可能性に着目して説明せよ。

問3

`ss` の出力にあるプロセスのPIDと、systemdのMain PIDを照合する理由を説明せよ。

章末解答

問1

答え： 例えば、サーバープログラムが想定とは別のアドレスやポートで待ち受けている場合。または、サービスの実行ユーザーにコンテンツファイルの読み取り権限がなく、要求を処理できない場合である。

問2

答え： `127.0.0.1:8081` は同じホスト内からの接続を受け付ける。`0.0.0.0:8081` は、そのホストのすべてのIPv4ローカルアドレスで待ち受ける。ただし、外部から実際に到達できるかはファイアウォールやネットワーク構成にも左右される。

問3

答え： 待受ソケットを利用しているプロセスが、目的のサービスとしてsystemdに管理されているプロセスかを確かめるためである。ただし、複数プロセスで動くサービスでは、待受を持つPIDがMain PIDと異なることもある。

参考資料

- Linux man-pages, socket(7)（2026-09-18確認）
- Linux man-pages, connect(2)（2026-09-24確認）
- curl, How to use curl（2026-09-24確認）
- Ubuntu 24.04実機の`man ss`、`man curl`、`man journalctl`（制作時に確認）
- 公開Lab v1.0.0のP5/M5 program、unit、checker（Lab固有値の正本）

第12章 SSHによる遠隔操作とファイル転送

12.1 接続元と接続先

離れたコンピューターを操作するときは、コマンドを入力する側と、そのコマンドを受けて動く側を区別する。SSH接続を開始する側をローカル（local）、接続先をリモート（remote）と呼ぶ。この呼び方は、操作する人の視点で決まる。ローカルが必ず手元の物理PCとは限らない。ブラウザで開いたクラウド上のターミナルからSSHを実行した場合、そのターミナルが動いている環境が接続元になる。

```
コマンドを入力する環境（ローカル）
        ↓ SSH接続
コマンドを実行するサーバー（リモート）
```

接続前後で、ユーザー名、ホスト名、カレントディレクトリが変わることがある。例えば、接続元で `id -un`、`hostname`、`pwd` を実行すると接続元の状態が分かる。SSHでログインした後に同じコマンドを実行すると、今度は接続先の状態が分かる。`exit` でリモートのシェルを終了すると、元のローカルのシェルに戻る。プロンプトの見た目だけでは、どちらの環境にいるか判断しない。

12.2 SSHが守るもの

SSH（Secure Shell）は、離れたコンピューターとの間に暗号化された通信経路を作り、リモートのシェルやコマンドを利用できるようにする仕組みである。接続時には、主に次の二つの確認を行う。

1. 接続先の確認（ホスト認証）：接続先が意図したサーバーかを、サーバーのホスト鍵で確認する。
2. 利用者の確認（ユーザー認証）：接続するユーザーがログインを許可されているかを確認する。公開鍵認証では、利用者の鍵ペアを使う。

その後、操作コマンド、実行結果、転送データは暗号化されたSSH接続を通る。ただし、暗号化だけでサーバーに到達できるわけではない。接続元から接続先まで通信できる経路も必要である。また、SSHにログインできても、接続先のファイルを自由に読んだり変更したりできるわけではない。ログイン後の操作には、接続先OSのユーザーとアクセス権が適用される。



図12-1 SSHでは、接続先を確かめるホスト鍵と、利用者を確かめるユーザー鍵が別の役割を持つ。

12.3 ユーザー鍵とホスト鍵の置き場所

公開鍵認証では、接続する利用者の秘密鍵（private key）をSSHクライアント側で保管する。対応する公開鍵（public key）は、接続先のユーザーがログインを許可する鍵として、通常は接続先の `~/.ssh/authorized_keys` に登録する。SSHクライアントは秘密鍵を相手へ渡さずに、その鍵を持っていることを証明する。

一方、ホスト鍵（host key）はサーバーを識別するための鍵である。サーバーはホスト秘密鍵を保持する。クライアントは接続先から提示されたホスト公開鍵を確認し、信頼した記録を通常は `~/.ssh/known_hosts` に保存する。初めて接続するときに表示される鍵の指紋は、信頼できる経路で得た情報と照合してから受け入れる。知らない鍵を無条件に受け入れると、接続先のなりすましに気づけない。

鍵の種類	主な目的	秘密鍵の保管場所	公開鍵に関する情報
ユーザー鍵	接続する利用者の認証	接続元	接続先の `authorized_keys`
ホスト鍵	接続先サーバーの確認	接続先	接続元の `known_hosts` など

秘密鍵を公開したり、Gitリポジトリへ追加したりしない。秘密鍵ファイルを他の利用者が読める状態にすると、SSHクライアントが使用を拒否することもある。なお、表の `~` はそれぞれの環境のユーザーのホームディレクトリを表す。同じ文字でも、接続元と接続先では指す場所が異なる。

12.4 接続名と通信経路

SSHクライアントは、接続先ごとの設定を接続元の `~/.ssh/config` に記述できる。次は設定の一例である。

```
Host study-server
  HostName server.example.edu
  User learner
  IdentityFile ~/.ssh/id_ed25519
```

この設定がある場合、`ssh study-server` と入力すると、SSHクライアントは `study-server` に対応するホスト名、ユーザー名、秘密鍵ファイルを使う。`Host` の値はSSH設定内の別名（ホストエイリアス）であり、DNSに登録されたホスト名とは別である。そのため、SSHでは使える別名が、SSH設定を読まない別のプログラムでは名前解決できないことがある。

接続先が直接到達できないネットワークにある場合は、踏み台や中継プログラムを経由してSSHの通信を届ける構成もある。OpenSSHでは `ProxyJump` や `ProxyCommand` などで経路を指定できる。これらは「どの経路で接続先へ届くか」の設定であり、「誰としてログインできるか」を決めるユーザー認証とは別である。正しい秘密鍵があっても、通信経路がなければ接続は成立しない。

12.5 対話ログインとリモートコマンド

`ssh study-server` を実行すると、認証後に接続先の対話シェルを開ける。これに対し、SSHコマンドの後ろに実行内容を付けると、接続先でその内容だけを実行し、結果を接続元へ返せる。

```
ssh study-server 'id -un; hostname; pwd'
```

この例の `id -un`、`hostname`、`pwd` は接続先で実行される。コマンド文字列を囲む外側のシングルクォートは、接続元のシェルが `$HOME` や `$(...)` を先に展開しないようにするためにも使える。たとえば `ssh study-server 'printf "%s\n" "$(hostname)" "$HOME"'` では、接続先のシェルがホスト名とホームディレクトリを調べる。外側をダブルクォートにすると、接続元のシェルが先に展開する。

`~` も、どちらのホスト・ユーザーのシェルが解釈するかで指すホームディレクトリが変わる。接続元の `~/notes.txt` と接続先の `~/notes.txt` は、同じ名前でも別のファイルである。

```
ssh study-server 'printf "%s\n" "$HOME" > "$HOME/home-path.txt"'
ssh study-server 'printf "%s\n" "$HOME"' > local-result.txt
```

一行目では `$HOME` の展開と `>` による保存が接続先で行われ、ファイルは接続先のホームディレクトリに作られる。二行目ではクォートの外にある `>` を接続元のシェルが解釈する。接続先から返った出力が、接続元の `local-result.txt` に保存される。どちらのシェルが文字列を解釈するかによって、同じ記号でも結果が変わる。

12.6 SCPでファイルを送受信する

scp はSSH接続と認証を利用してファイルを転送するコマンドである。基本形は scp 転送元 転送先 であり、接続先のパスは ホスト名:パス と書く。

```
scp notes.txt study-server:~/notes.txt
scp study-server:~/report.txt report.txt
```

一行目は接続元の notes.txt を接続先へ送るアップロードである。二行目は接続先の report.txt を接続元へ受け取るダウンロードである。コロン : の後ろの ~ は接続先ユーザーのホームディレクトリを指す。study-server のようなSSH設定の別名は scp でも使える。

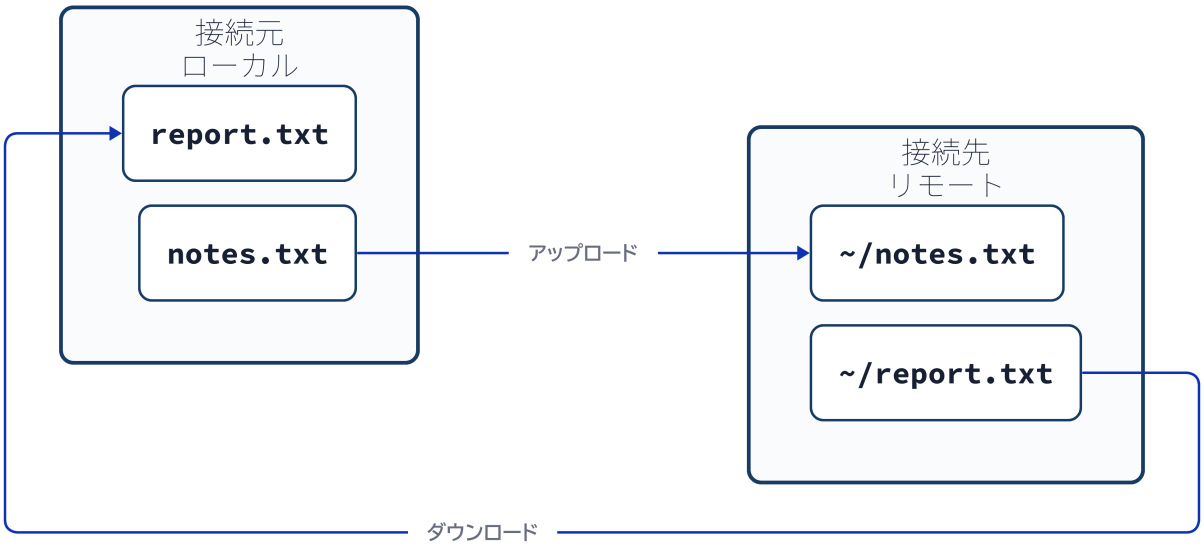


図12-2 コマンドを実行する側を基準に、アップロードとダウンロードの方向を区別する。

転送元と転送先を逆にすると、意図とは反対の方向へファイルをコピーする。転送先に同名のファイルがある場合は上書きにも注意が必要である。接続元と接続先のファイルは、それぞれ別のファイルシステム上に存在する。

12.7 内容の一致とファイルの属性

転送後に二つのファイルの内容が同じかを確かめるには、両方のファイルで sha256sum を実行し、ハッシュ値を比較できる。

```
sha256sum notes.txt
ssh study-server 'sha256sum ~/notes.txt'
```

ハッシュ値の一致は、二つのファイルの内容が同じであることを確認する強い根拠になる。一方、所有者、グループ、アクセス権モード、保存場所はハッシュ計算に含まれない。転送先では別のユーザーがファイルの所有者になったり、アクセス権モードが異なったりする場合がある。これらの属性は接続先で stat を使って別に確認する。

```
ssh study-server 'stat -c "%U:%G %a %n" ~/notes.txt'
```

内容の確認と属性の確認は、答える問いが違う。sha256sum は「中身が同じか」、stat は「接続先で誰のファイルとして、どの権限で置かれているか」を調べる。

章末問題

問1

ブラウザで開いたクラウド上のターミナルから別のサーバーへSSH接続する。このとき、ローカルとリモートはそれぞれどの環境か。

問2

SSHのユーザー鍵とホスト鍵は、それぞれ何を確認するために使うか。また、ユーザー秘密鍵はどちらに置くか。

問3

SSH設定のホストエイリアスとDNSホスト名はどう違うか。正しい鍵を持っていても接続できない場合がある理由も述べよ。

問4

次の二つのコマンドでは、`$HOME` の展開と結果の保存はそれぞれどちらの環境で行われるか。

```
ssh study-server 'printf "%s\n" "$HOME" > "$HOME/home-path.txt"'
ssh study-server 'printf "%s\n" "$HOME"' > local-result.txt
```

問5

`scp` で転送した二つのファイルのSHA-256ハッシュ値が一致した。所有者とアクセス権モードも同じだと言えるか。

章末解答

問1

答え： SSHコマンドを実行するクラウド上のターミナル環境がローカルで、接続先のサーバーがリモートである。ブラウザを表示する物理PCが必ずローカルになるわけではない。

問2

答え： ユーザー鍵は接続する利用者を認証し、ホスト鍵は接続先サーバーを確認する。ユーザー秘密鍵は接続元に保管し、接続先へ渡さない。

問3

答え： ホストエイリアスは接続元のSSH設定で使う別名であり、DNSホスト名とは限らない。鍵が正しくても、接続先まで通信できる経路がなければSSH接続は成立しない。

問4

答え： どちらも `$HOME` は接続先で展開される。一行目の `>` は引用符の内側にあるため接続先で保存する。二行目の `>` は引用符の外側にあるため接続元で保存する。

問5

答え： 言えない。ハッシュ値はファイルの内容を比較するものであり、所有者とアクセス権モードは含まない。これらは転送先で `stat` などを使って別に確認する。

参考資料

- Ubuntu Server documentation, OpenSSH server（2026-09-24確認）
- OpenBSD manual pages, ssh(1)、ssh_config(5)、scp(1)（2026-09-24確認）
- GNU Coreutils, sha256sum(1)（2026-09-24確認）

本書のまとめ

コンピューター上のプログラムは、OSが提供する機能を利用して動く。Linuxを操作するときは、コマンドの名前だけでなく、その操作がファイル、ユーザー、プロセス、通信のどれに関わるかを考えると、結果を理解しやすい。

設定ファイルが存在すること、サービスのプロセスが動いていること、ネットワークから期待した応答を得られることは、それぞれ異なる状態である。ある一つの状態が正しくても、ほかの状態が正しいとは限らない。ファイルの所有者やアクセス権、サービスの実行ユーザー、待受ソケット、ログを区別して読むことが大切である。

後続の授業では、Nginx、BIND 9、PostgreSQLなど、用途の異なるソフトウェアを扱う。それぞれの設定方法は異なるが、「どのユーザーで動くか」「どのファイルを使うか」「どこで通信を受けるか」「動作をどこで確認できるか」という問いは共通して使える。本書で学んだOSの基本概念を、新しいシステムを理解するための土台にしてほしい。

コマンドの綴りやオプションを読み直したい場合は、巻末の参照資料として用意した[用語集・コマンド早見表](../reference.md)の「コマンドの読み方」「findを段階的に読む」「grepでファイルの中の行を探す」「tailで末尾の行を読む」を参照してほしい。`id -un`のuser・nameの対応や、検索開始位置・条件・表示を分けた読み方を、短い実行例で説明している。